

UNIVERSITÀ
DEGLI STUDI
DI PADOVA

MAPD-B PROJECT

ANOMALY DETECTION AND PREDICTIVE MAINTENANCE

GROUP 5

CALCAGNI - LAVARDA - ANDREACCHIO - GONZATI

The cluster, hosted by the **CloudVeneto** infrastructure, is composed by 4 *cldarepd.medium* VMs, with 4 GB of RAM and 2 vCPUs:

- **1 Master node:** cluster manager and orchestrator (hosts Spark Master & Driver)
- **3 Slave nodes:** dedicated computing resources (6 vCPUs in total)

The default configs used are:

- **Shuffle partitions:** spark's default of 200
- **Allocated memory:** 1.5GB for the driver and 2GB for the executors
- **Storage & I/O:** CloudVeneto Object Store, built on Ceph and compatible with AWS S3

The used dataset contains data from four different devices and has four columns:

- **when**: UNIX timestamp
- **hwid**: device identifier
- **metric**: name of the metric
- **value**: value of the metric

All data have a different sampling frequency (spanning from 30s to 1min), which will be normalized before conducting the analysis.

When	Hwid	Metric	Value
1601510485159	SW-065	SA4	0
1601510485159	SW-065	SA3	0
1601510485159	SW-065	SA2	0
1601510485159	SW-065	S34	0
1601510485159	SW-065	S33	1
1601510485159	SW-065	S25	0
1601510485159	SW-065	S19	0
1601510485159	SW-065	S17	0
1601510485159	SW-065	S16	0
1601510485159	SW-065	S15	0
1601510532889	SW-115	S41	153
1601510535401	SW-115	S143	100
1601510563238	SW-115	S41	152
1601510565967	SW-115	S143	100
1601510593606	SW-115	S41	153
1601510596120	SW-115	S143	100
1601510624047	SW-115	S41	152
1601510626780	SW-115	S143	100
1601510654443	SW-115	S41	152
1601510657046	SW-115	S143	100

Example of the data

The first task for *Predictive Maintenance* is the identification of faulty alarms:

- **Metrics:** A5 and A9 (alarms on Circuit 1 and 2 respectively)
- **Conversion:** stored as 16-bit integer in *little-endian format*, converted to 16-bit strings, in which the n-th bit indicates the output of the n-th sensor, starting from the LSB.
- **Bits position:** the examined bits were in positions 6, 7 and 8.
- **Overheating:** if 1+ bit(s) were equal to 1 in either one or both metrics, the device was “marked” by a *is_overheated* bool.

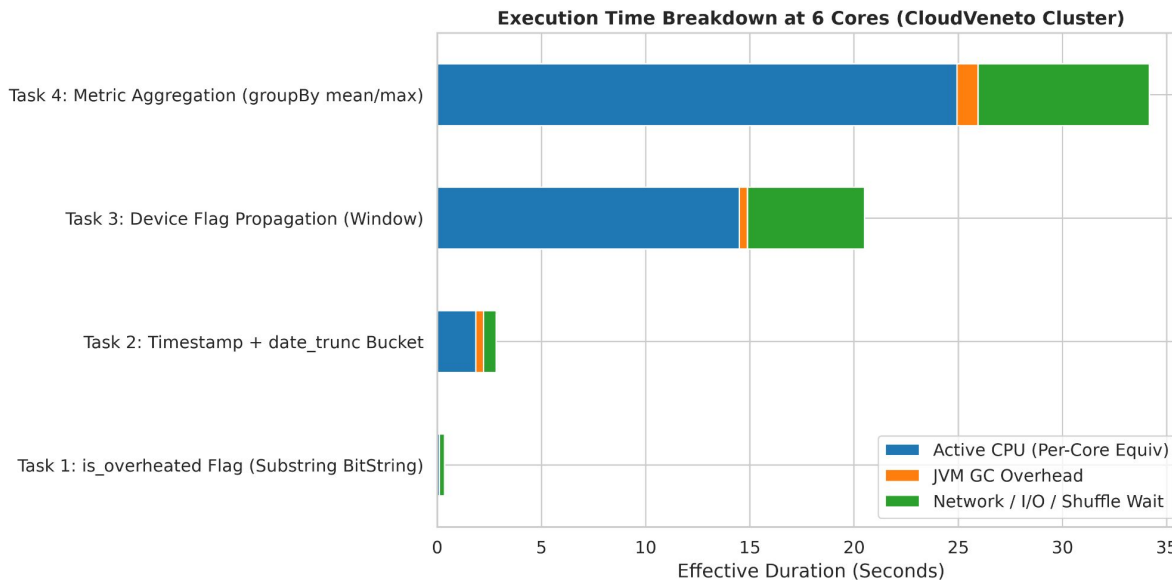
This was done before the sampling frequency normalization, to avoid losing information on the alarms by averaging discrete values.

After the conversion and identification of the alarms, data sampling frequency was normalized to **1 minute** creating a *timestamp_bucket*.

To aggregate the metrics in each “bucket”, the nature of the data was considered:

- **Engine Status Metrics (S117, S118, S169, S170):** these metrics indicate the power status (ON/OFF) of the engine, therefore they were aggregated by their maximum value.
- **Discrete states, binary flags and alarms:** they were aggregated by their maximum value, to preserve any alarm event.
- **Continuous physical measurements (e.g. temperatures):** they were aggregated by their mean value.
- **Cumulative energy counters:** they were aggregated by their maximum value, reflecting the total accumulated energy.

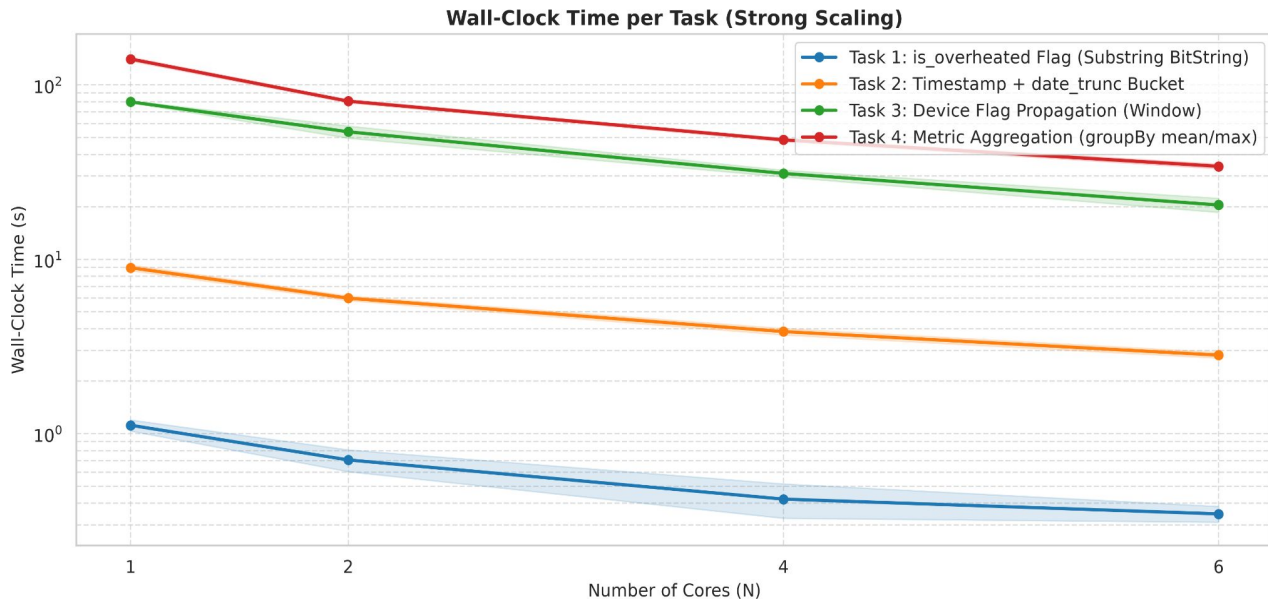
Decomposition of the execution time of the notebook at 6 cores.



- **Dominant workloads:**
Tasks 3 & 4 accounts for ~97% of the total execution time.
- **GC overhead:** negligible across all tasks (being ~1s in task 4).
- **Shuffle & I/O overhead:**
Wide transformations in tasks 3 & 4 introduce measurable network and shuffle wait

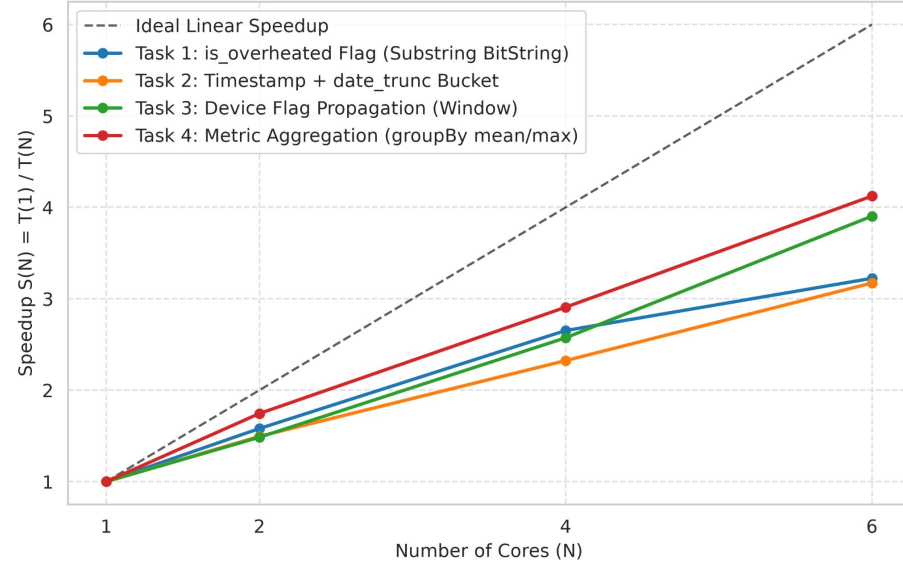
Evaluation of wall-clock time per task with $N \in \{1, 2, 4, 6\}$ cores.

5 runs per task, to get mean $\pm \sigma$.

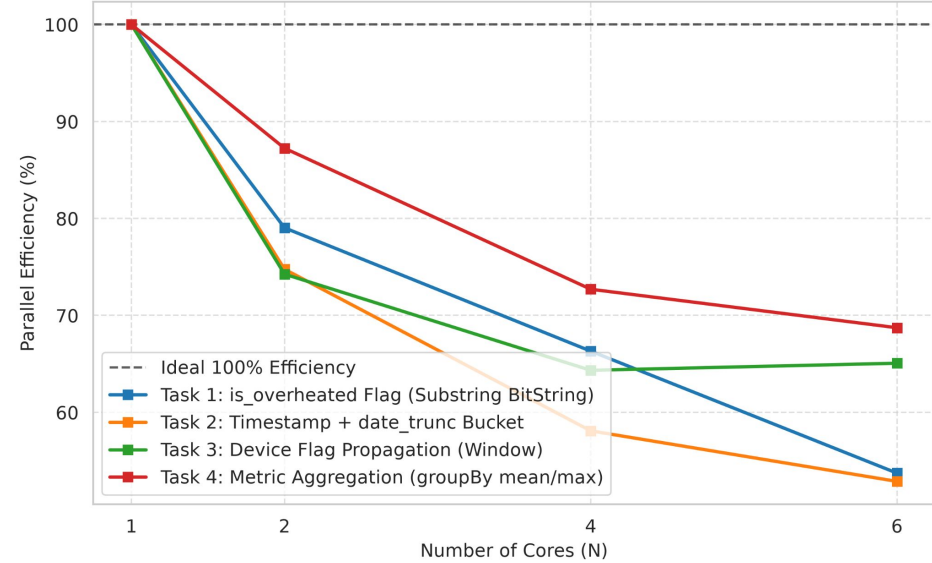


- **Task 1 & 2 (narrow):**
“map-like” operations, very fast and almost ideal scaling (due to no shuffling).
- **Task 3 & 4 (wide):**
“group-like” operations, slower due to shuffling, with task 4 being the bottleneck.

Strong Scaling Speedup $S(N)$ across Spark Computational Tasks

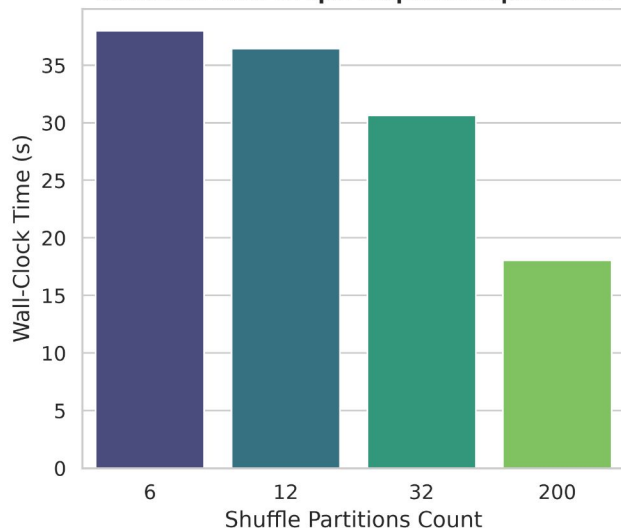


Parallel Efficiency $E(N) = S(N)/N$ across Spark Tasks

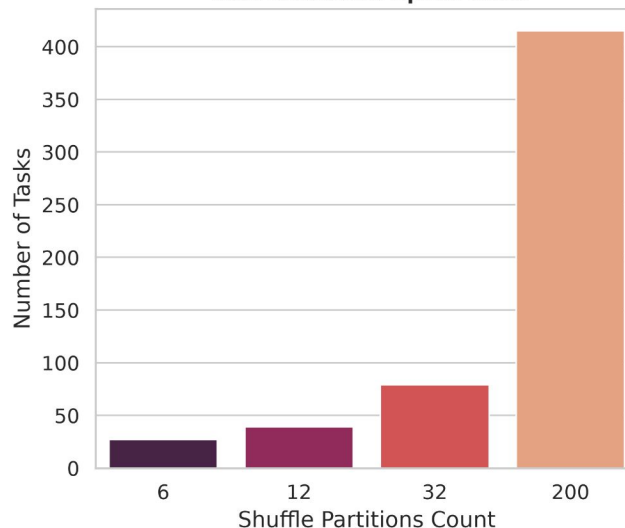


Heavier tasks (Tasks 3 & 4) achieve the highest speedup and parallel efficiency (at 6 cores). With the execution time of lighter tasks (Tasks 1 & 2) being of a few seconds, the **CPU utilization** becomes **negligible** compared to the **fixed overheads** (like task scheduling).

Execution Time vs spark.sql.shuffle.partitions

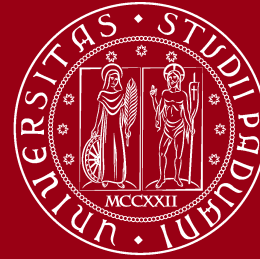


Total Scheduled Spark Tasks



Evaluation of execution time for Task 4 at N=6 cores with $p \in \{6, 12, 32, 200\}$ partitions, with $p=200$ being the default value:

- **Execution time:** decreases from ~37s to ~18s.
- **Granularity:** smaller partitions reduce data volume and memory pressure.
- **Scheduling overhead:** despite the rise in the number of tasks, task coordination overhead is negligible compared to the time gains.



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

ANOMALY DETECTION

PART 1

Distributed analysis on the energy high frequency
and its correlations with other metrics

Original Data Frame for this section

COLUMN NAME	DESCRIPTION
<code>timestamp_bucket</code>	1 minute timestamp
<code>hwid</code>	device identifier
<code>metric</code>	type of measure
<code>aggregated_value</code>	Either maximum or mean value of the bucket

High-Frequency Research - Approach

- **Problem:** Defining high frequency
- **Method:** Group measurements by 1-hour timestamps and calculate engine switch counts
- **Answer:** Statistical definition of high frequency

Main methods /
functions:

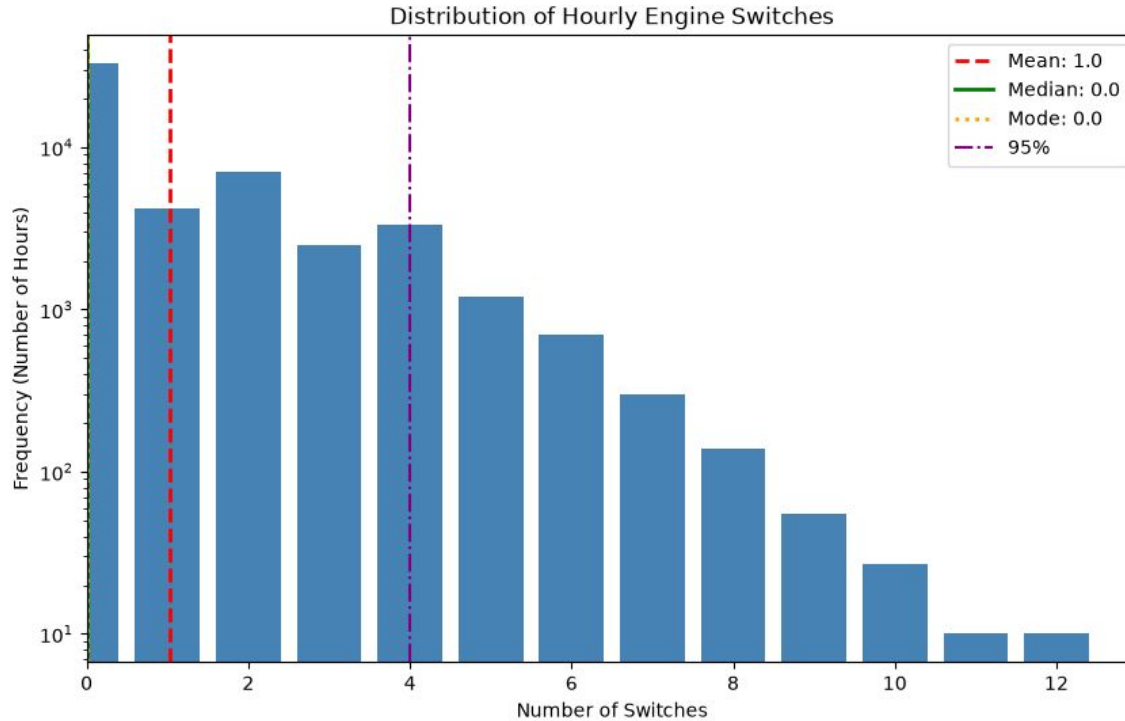
- `groupBy`
- `coalesce`
- `date_trunc`
- `lag`

High-Frequency Research - Approach

Final Data Frame structure

COLUMN NAME	DESCRIPTION
<code>hour_bucket</code>	1 hour timestamp
<code>hwid</code>	device identifier
<code>metric</code>	type of measure
<code>total_state_switches</code>	total state switches count

High-Frequency Research - Result



total_count	52200
min	0
max	12
mean	1.02659
stddev	1.651326
median	0
95% percentile	4
mode	0

Correlation analysis - Approach

- **Problem:** Correlate engines high switch frequency with the other metrics
- **Method:** Divide the metrics into subgroups and correlate them with the engines switches
- **NB:** Device must be kept separated

Main methods / functions:

- `groupBy`
- `pivot`
- `join`
- `corr`

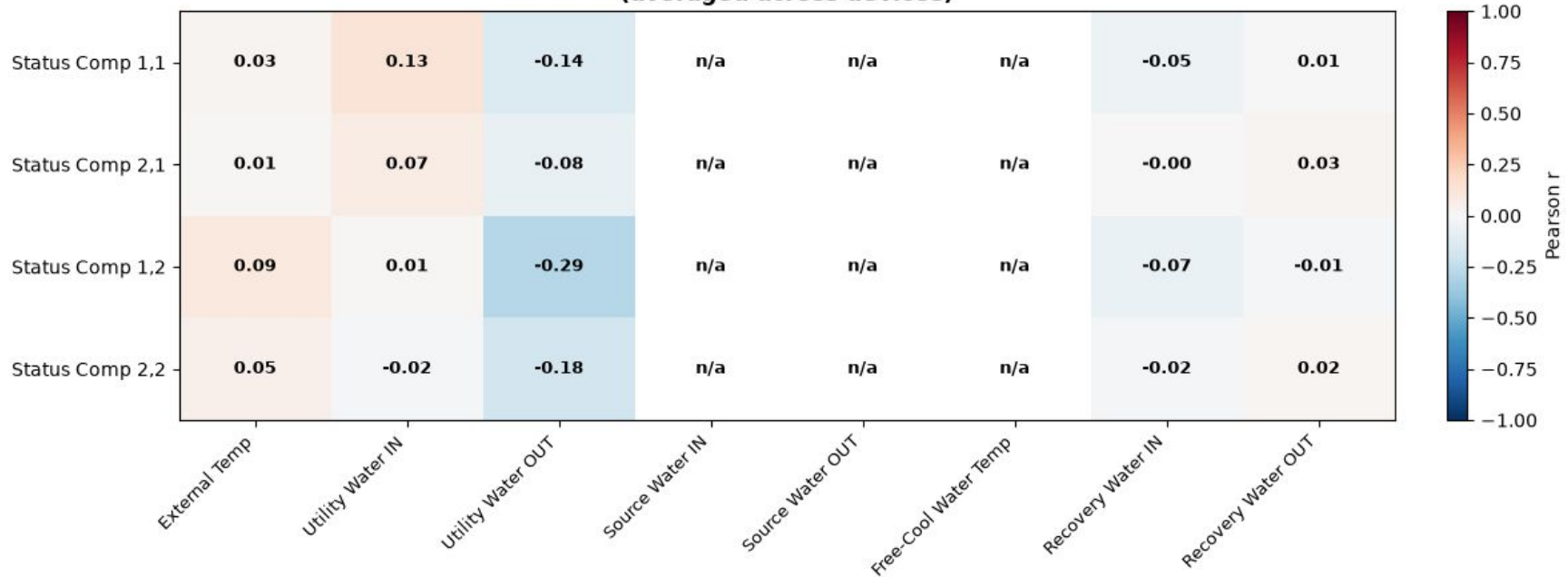
Correlation analysis - Approach

The procedure is organized into classic python functions:

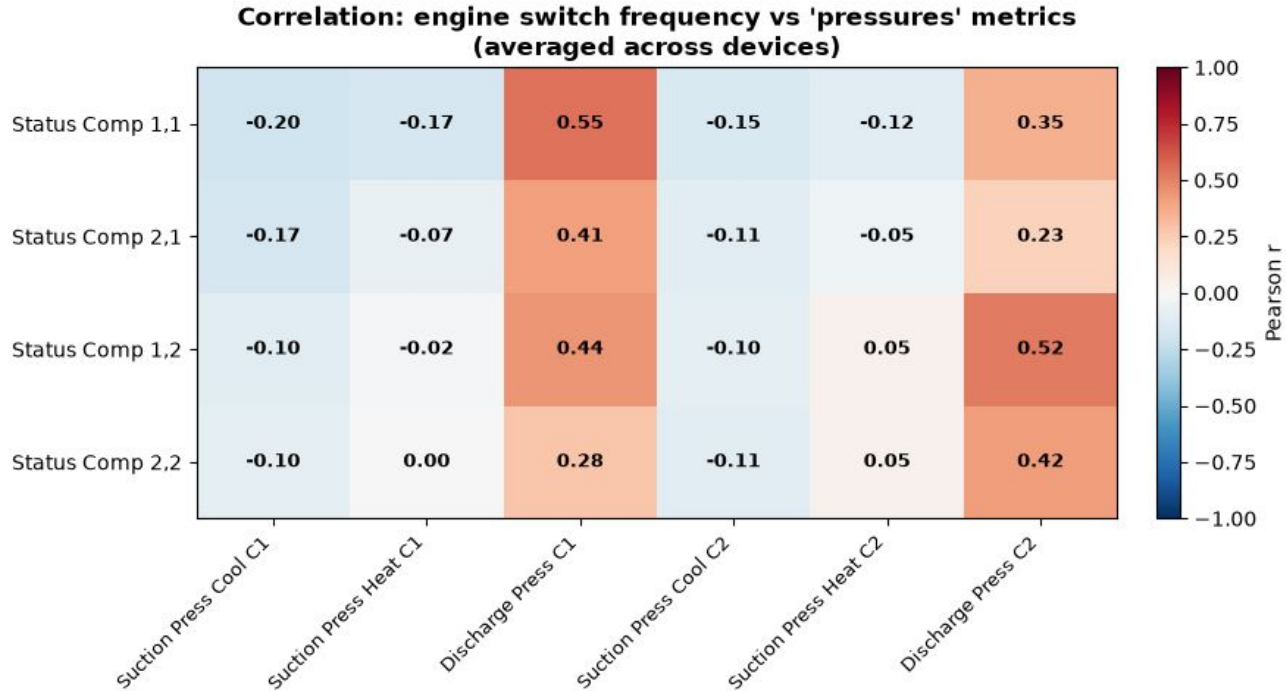
- ***engines_switch_counts_df*** -> Creates a new Data Frame with as columns: hour_bucket, hwid, engine metrics and target metrics subgroup (both pivoted)
- ***corr_df*** -> Creates a new Data Frames with as columns: hwid and correlations engine_switches - target_metrics
- ***engine_sensors_heatmap*** -> Creates the ultimate Data Frame and corresponding heatmap, with correlations averaged across the devices (hwid)

Correlation analysis - Results

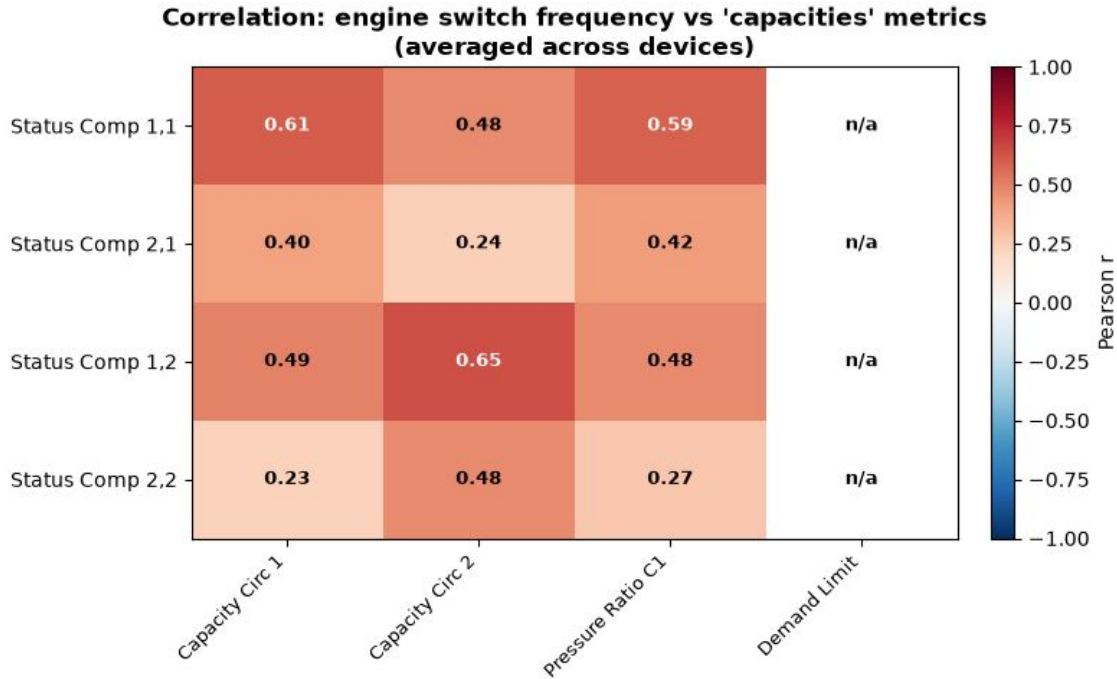
Correlation: engine switch frequency vs 'temperatures' metrics
(averaged across devices)



Correlation analysis - Results

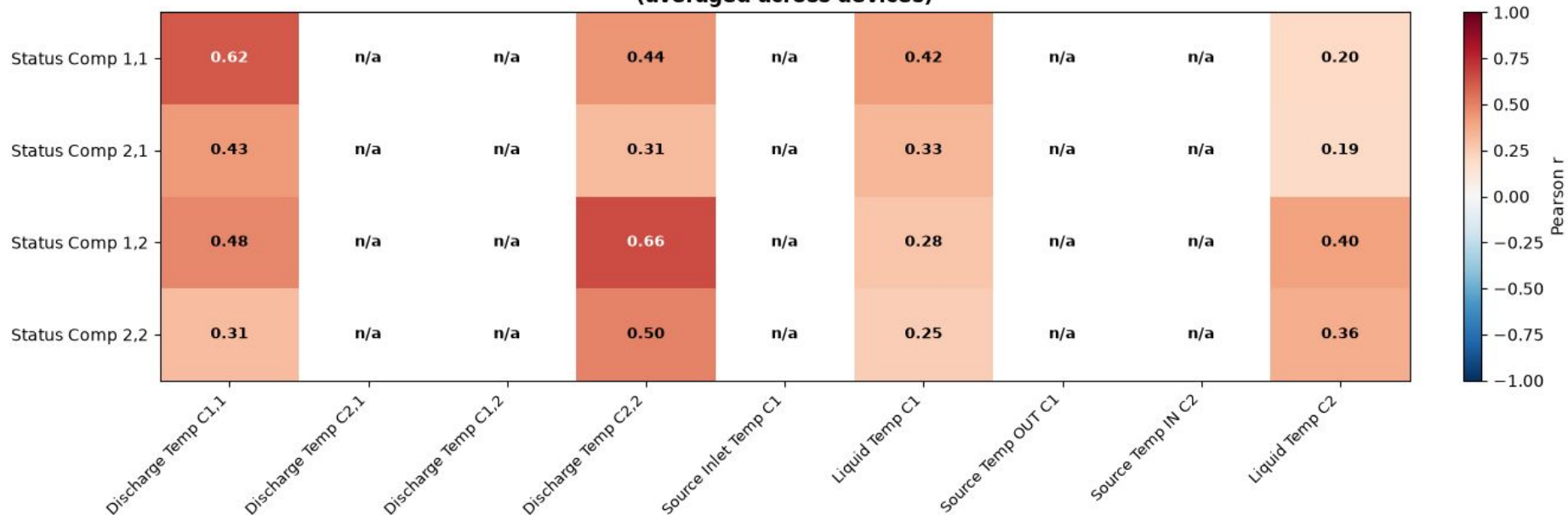


Correlation analysis - Results

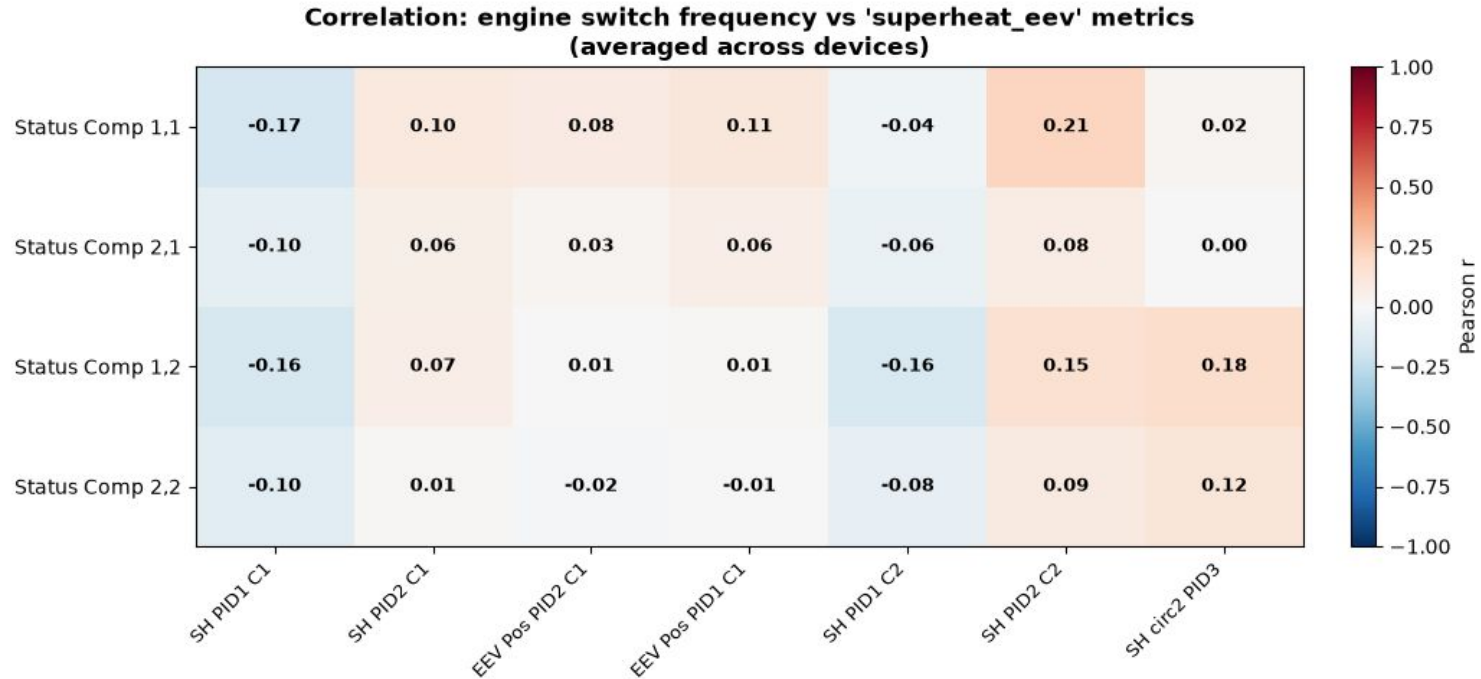


Correlation analysis - Results

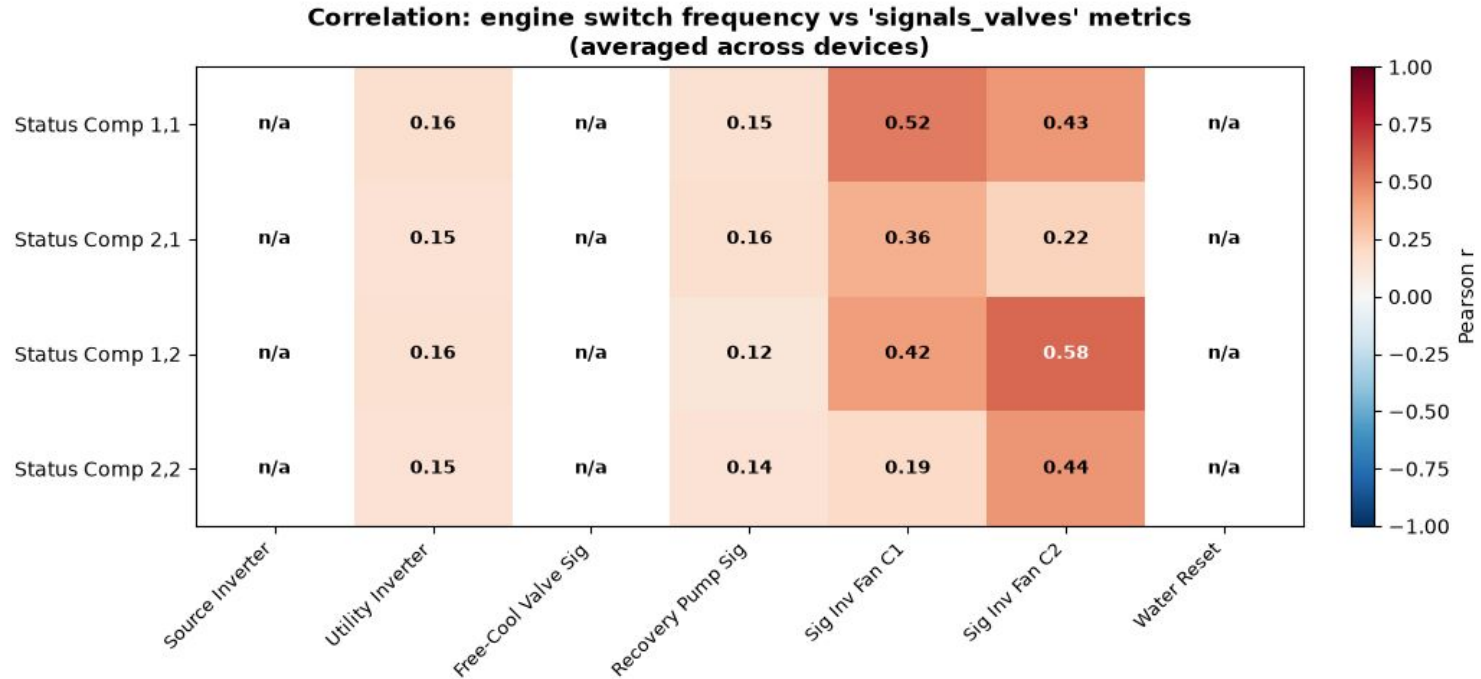
Correlation: engine switch frequency vs 'discharge_temps' metrics
(averaged across devices)



Correlation analysis - Results

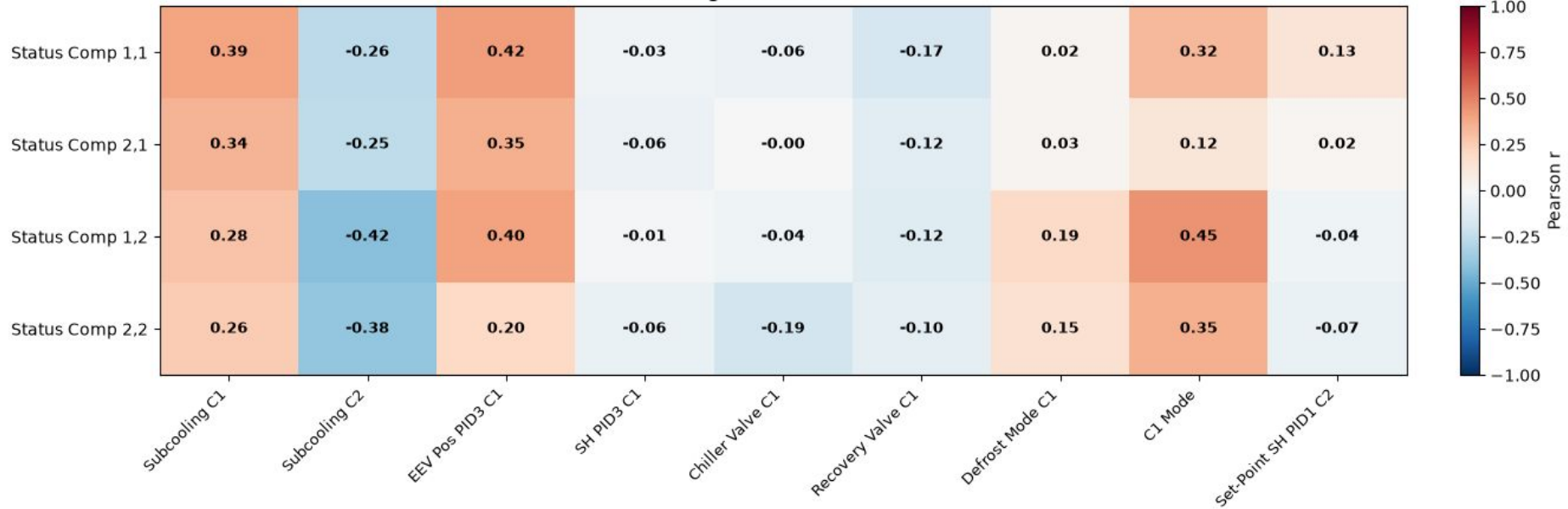


Correlation analysis - Results



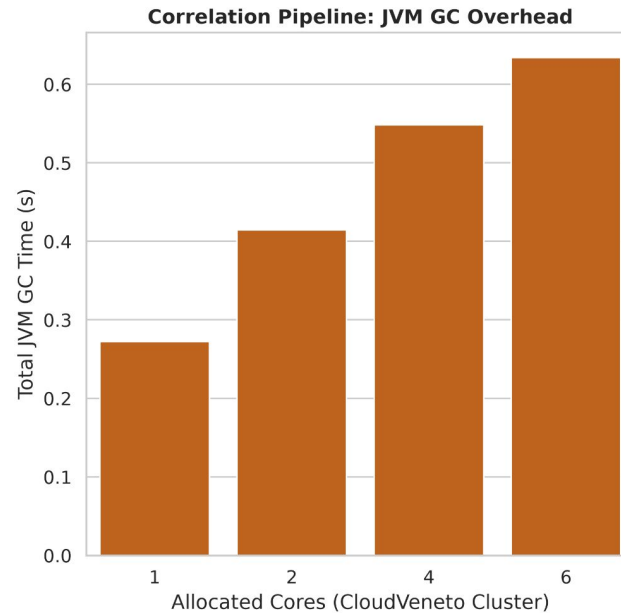
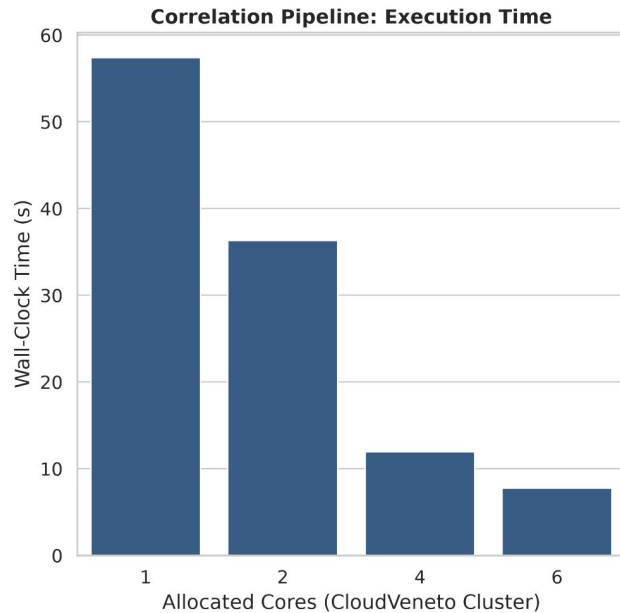
Correlation analysis - Results

Correlation: engine switch frequency vs 'subcooling_misc' metrics
(averaged across devices)



Benchmarks Analysis - Correlation pipeline

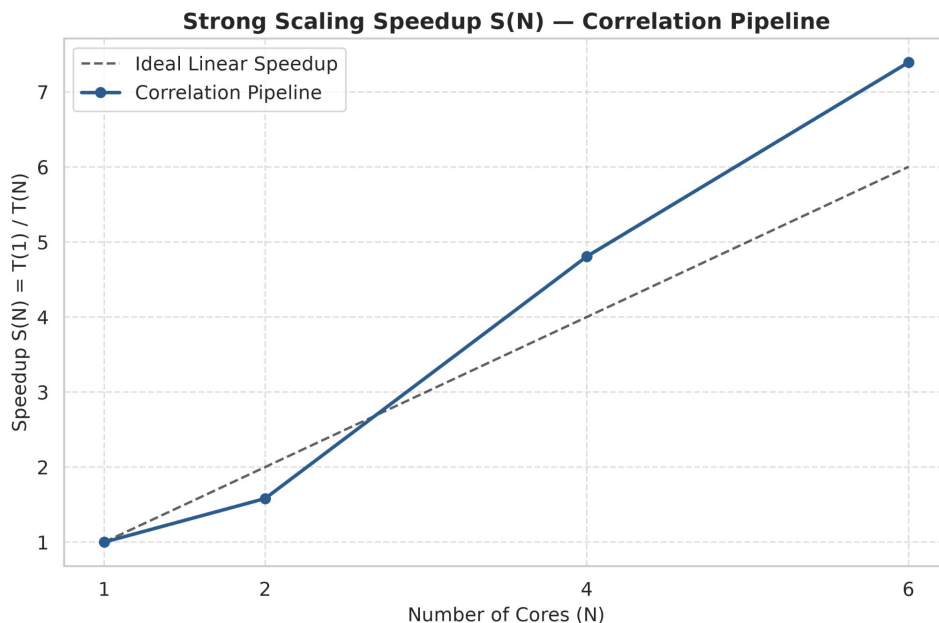
Execution Time and GC Time



1. The execution time scales incredibly fast with the number of cores
2. GC Time seems reasonable, but strange for the dataset dimension

Benchmarks Analysis - Correlation pipeline

Strong Scaling Speed-Up



This confirms with no doubt the strangeness of the previous slide

Either I found a new law... or there is a serious problem with the measured times

Benchmarks Analysis - Correlation pipeline

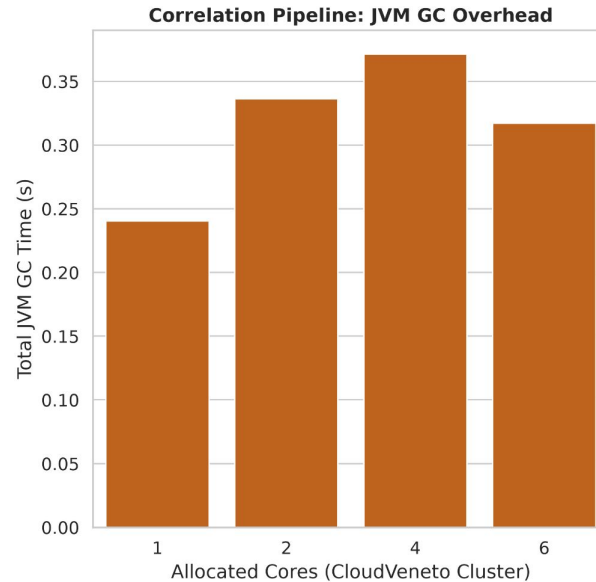
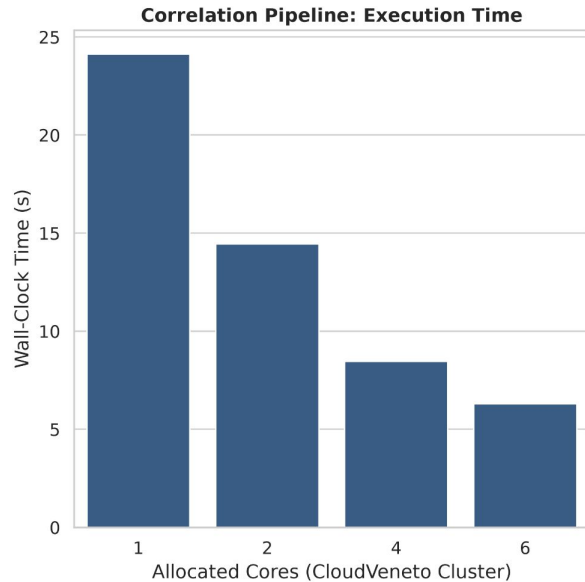
The problem

I tested the pipeline repeatedly to get consistent results. This created the following:

1. First run: the code must be compiled, and the execution plan is built. This triggers the **JIT Compiler**, which obviously needs time. The first measurement is strongly altered by this
2. Other runs: do not need this since the pattern is the same, thus the execution is much faster
3. This problem gets amplified as the number of cores grows

Benchmarks Analysis - Correlation pipeline

Execution Time and GC Time

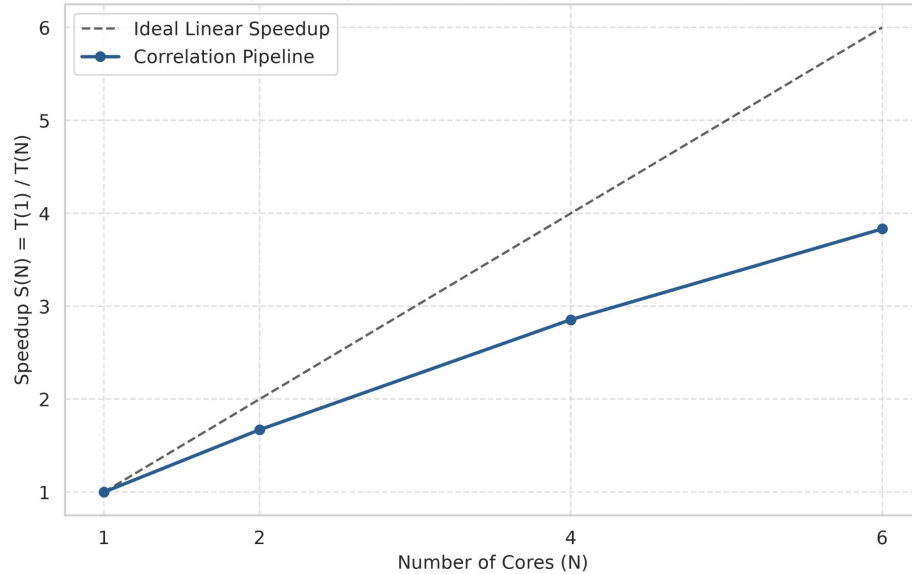


I thus excluded the first run, and the results became realistic

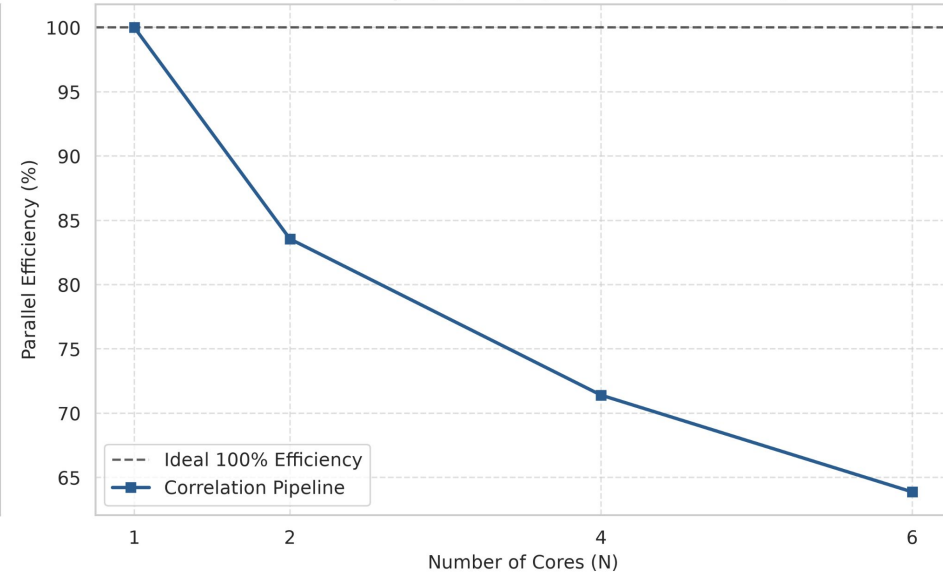
Benchmarks Analysis - Correlation pipeline

Strong Scaling Speed-Up

Strong Scaling Speedup $S(N)$ — Correlation Pipeline

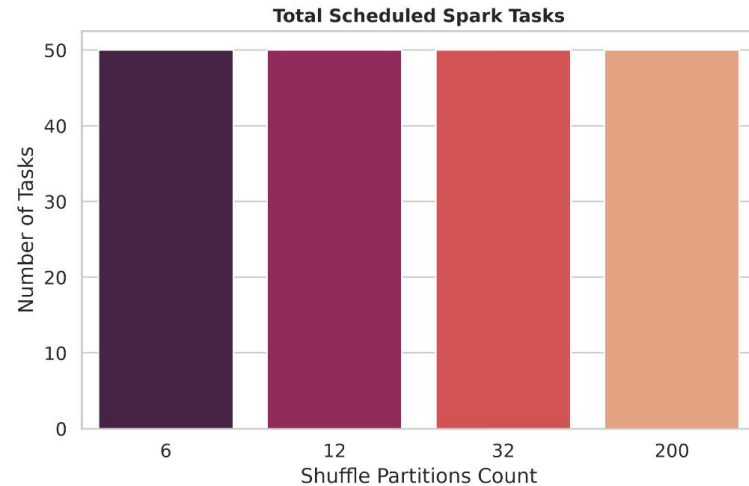
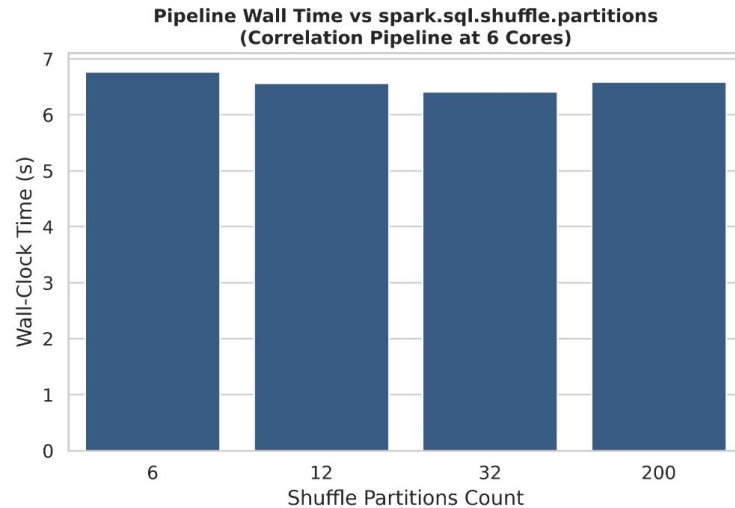


Parallel Efficiency $E(N) = S(N)/N$ — Correlation Pipeline



Benchmarks Analysis - Correlation pipeline

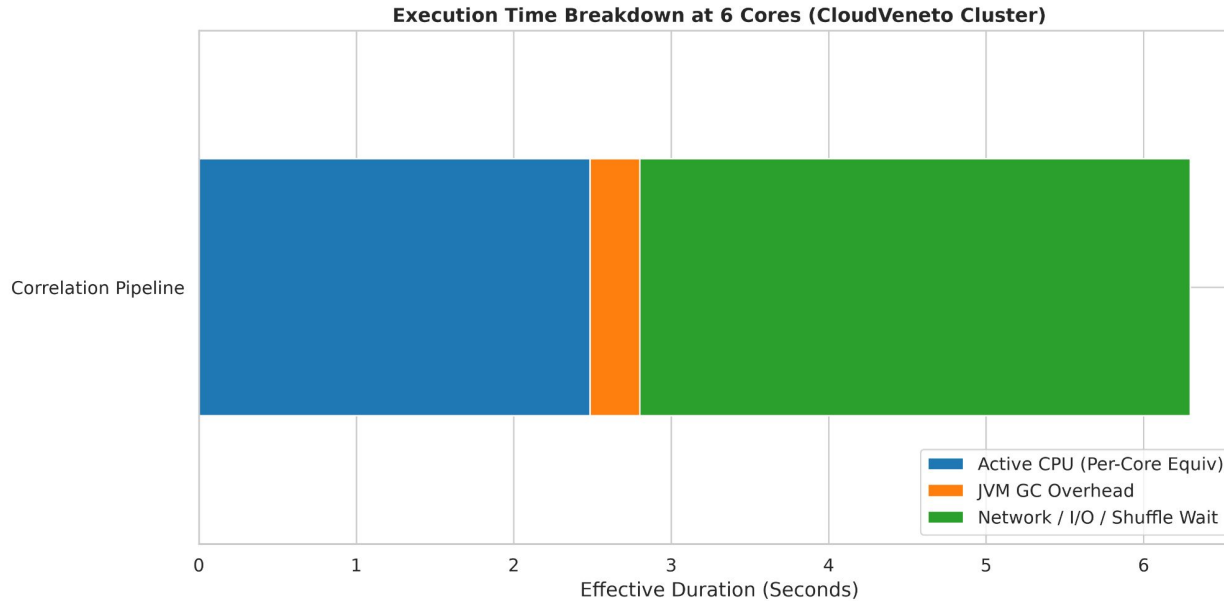
Shuffle partitions

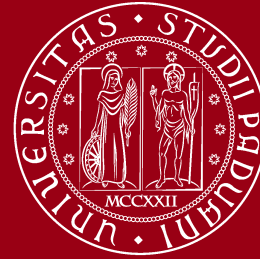


NB: The number of tasks is constant due to the very small partition size

Benchmarks Analysis - Correlation pipeline

Execution time division through operations type





UNIVERSITÀ
DEGLI STUDI
DI PADOVA

ANOMALY DETECTION

PART 2

Distributed analysis on the energy high frequency
and its correlations with other metrics

Section 3.3.2 asks for one thing: correlations between device loading percentage and external temperature.

Variable	Meaning	Unit	Implicit decimals
S125	Capacity, circuit 1	%	0
S181	Capacity, circuit 2	%	0
S41	External temperature	°C	1

Why the decimals matter

Values are stored as integers. S41 carries one implicit decimal: raw 153 means 15.3 °C. Skip the rescale and every temperature is ten times too large. Rescaled: -11 °C to +34 °C.

The machines

Four reversible heat pumps, each with two circuits (S125 and S181) and two compressors inside each circuit: four compressors per machine.

What the physics predicts

A heat pump moves heat rather than creating it. Compressor work grows with the thermal lift (the gap between outside air and the set-point) either way. So load should rise in the cold and again in the heat.

Raw data arrives long: one row per sensor reading. Correlating two quantities needs them on the same row.



671,850 device-minutes reach the wide table: four devices, six months, one row per minute with all three metrics side by side.

Two group-bys back to back: one to put every metric on the same minute, one to turn metrics into columns.

Sensors write at unsynchronized intervals

By truncating to the minute, we obtain a common key, and the average gives us the delivered capacity.

The explicit pivot list is an optimisation

Passing `t2_TARGET` saves Spark a separate job just to discover which distinct values exist, i.e. reading the data twice.

Why `cache()` is here

This is where the pipeline forks: `t2_df_wide` feeds 20+ downstream calculations. Without it each would redo the S3 read, filter, resample and pivot, shuffles included.

After rescaling: do the numbers still make physical sense?

-11 °C to +34 °C, loads strictly within 0–100 %, 2020-10-01 to 2021-03-31. Had the decimal rescale been skipped, this line would read a maximum of 340 °C.

One load figure per machine, and the hourly table everything downstream is computed on.

Mean load and gaps in the readings

Mean load = average of available circuit readings

Missing (**NULL**) circuit readings are excluded from the calculation.

~2% of minutes have one missing reading; the circuit itself is still present.

Aggregation per hour

Temperature drifts over hours; compressor load oscillates minute by minute with the control cycle. Correlating at the minute feeds in high-frequency noise that temperature cannot explain. Hourly puts both quantities on the same timescale.

$$\text{Load Avg.} = \frac{\sum \text{Reported Capacities}}{\# \text{ Reported Circuits}}$$

Before any correlation is computed, it is worth knowing what fraction of the data is a machine sitting still.

53.6 %

of all recorded minutes have both circuits at zero

671,850 minutes with known load
360,336 minutes at zero
311,514 minutes running

Why this number has to be stated

A correlation computed over all rows is then dominated by a question nobody asked: “is the machine on?” rather than “how hard is it working?”. With more than half the rows at zero, the first question drowns out the second, and nothing in the coefficient tells you that has happened.

It is normal, not a data problem

A thermal plant reaches its set-point and shuts off until the temperature drifts again. The zeros are real operating states, not missing values.

So every correlation in this talk is reported on both sets: all minutes, and running minutes only. Where the two disagree, that disagreement is the result.

If the preparation choices were hiding the relationship, the three columns would disagree. They do not.

	per minute	hourly	hourly, running only
s125	-0.140	-0.175	-0.238
s181	-0.089	-0.097	-0.105
load_avg	-0.137	-0.157	-0.213

The answer barely moves

Dropping idle minutes strengthens the coefficient from -0.14 to -0.21 , and hourly aggregation adds a little. The sign never changes and the magnitude stays small throughout.

So neither choice is what is hiding the relationship.

Read on its own, -0.16 would be reported as “essentially no relationship between load and outside temperature”. The next two sections are about why that reading is wrong, in two different ways.

Spark has no Spearman on `stat.corr`, but it is Pearson applied to ranks.

Spearman -0.09 vs Pearson -0.16

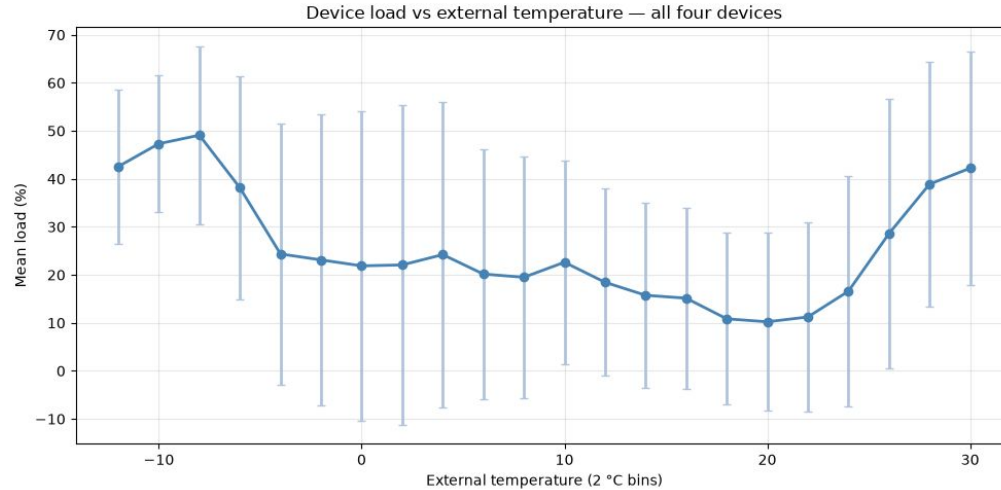
Spearman catches any monotone relationship, curved or not. Coming out smaller rules out “strong but bent one way”. What is left is a relationship that reverses.

These small coefficients show that the pooled relationship is neither linear nor monotonic.

Spearman's rank

Spearman's rank requires a single global ordering, forcing Spark to consolidate all rows into one partition and execute on a single core. While trivial on 13,052 hourly rows, this single-threaded constraint kills parallelism and drives the poor scaling seen in benchmark Phase 3 on raw data. `partitionBy("hwid")` cannot fix this, as it produces four isolated rankings.

Mean load per 2 °C bin, all devices pooled. Error bars are the standard deviation of the data inside each bin.



Mean load by bin

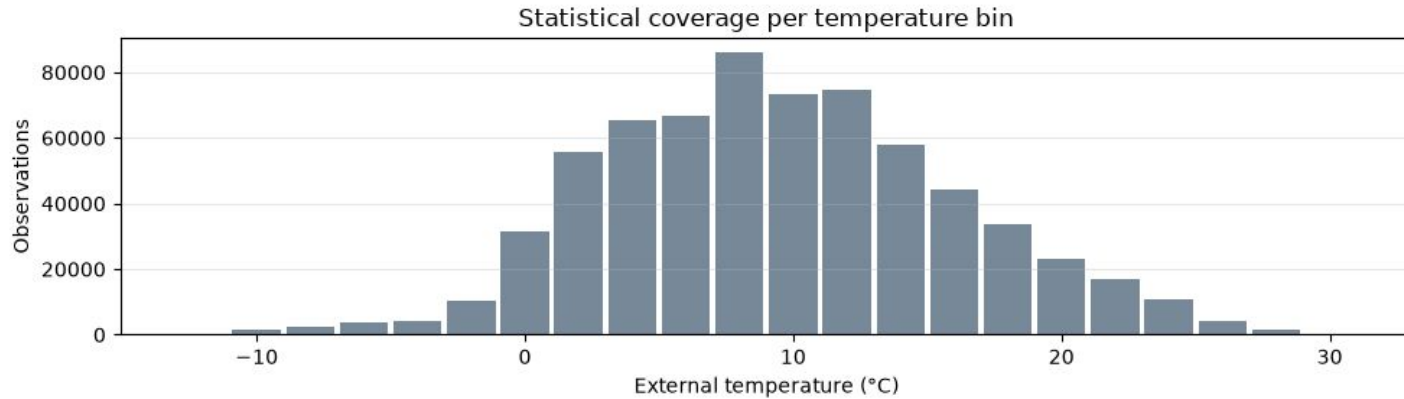
-8 °C	49.1 %
8 °C	19.5 %
18-20 °C	10.2 %
26 °C	28.6 %
30 °C	42.2 %

Minimum at 18-20 °C — plausibly the indoor set-point.

Why Pearson could not see it

A linear coefficient on a symmetric U measures the average slope, near zero by construction. The branches cancel.

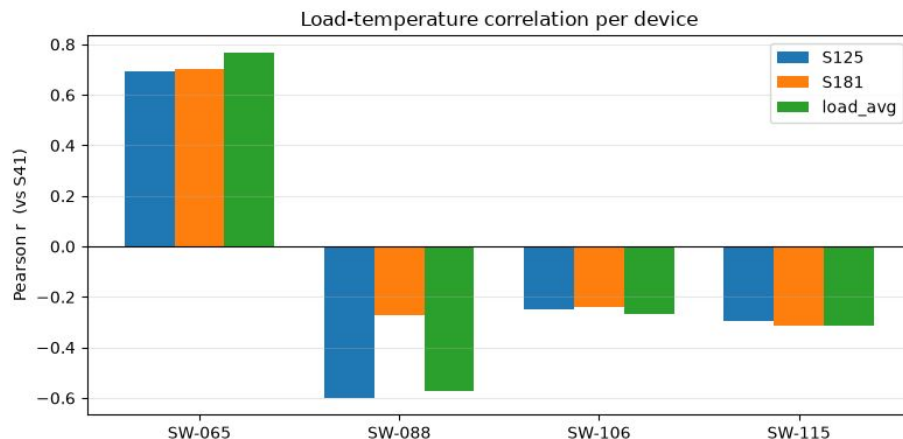
The error bars say how spread out the measurements are. They say nothing about how many there are.



Both ends of the 'U' are thin

Coverage peaks at 8 °C with 86,065 minutes, which is simply where a Veneto winter spends its time. The extremes carry far less: 4,260 minutes in the 26 °C bin, and only 2,220 at -8 °C. So the highest point of the U, 49.1 % load in the cold, is also its least supported one.

The same coefficient computed per device instead of over the pooled data.



Fleet: $r = -0.16$

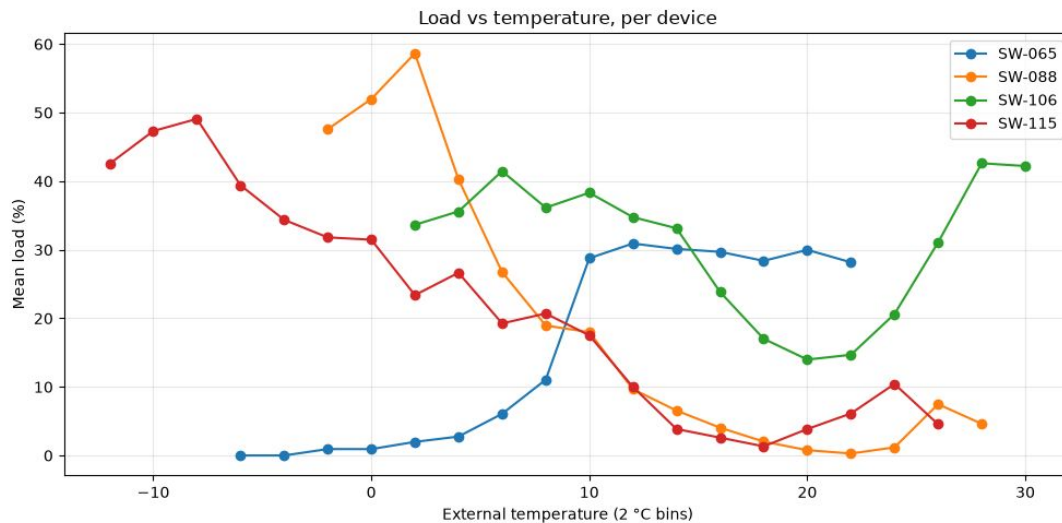
Reads as “almost no relationship”. It is a cancellation artefact.

● SW-065	+0.767	loads up as it gets warmer
● SW-088	-0.571	loads up as it gets colder
● SW-106	-0.269	weakest — see Result 3
● SW-115	-0.313	loads up as it gets colder

Simpson's paradox

A trend that reverses depending on whether you look at subgroups or the aggregate. The four per-device figures are the result; the fleet figure is not a summary of them.

The same binned profile, one line per device — the per-device picture of the coefficients on the previous slide.



● ● The cold branch

SW-088 and SW-115 produce the left-hand side. Their load collapses towards 20 °C.

● The warm branch only

SW-065 is off below 4 °C, so it contributes almost nothing to the cold side.

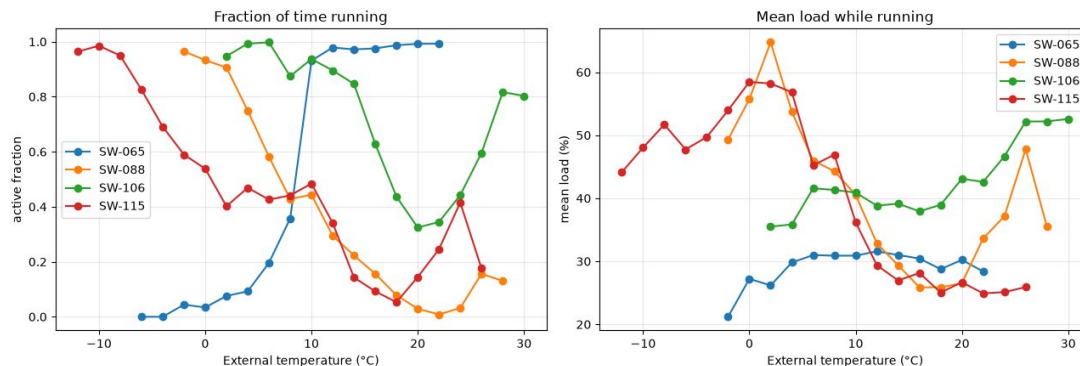
● Both branches

SW-106 is the only one that covers the whole range on its own, so it traces an approximate U by itself.

This is the caveat that keeps the pooled curve honest. The U is real and it is measured, but as a fleet aggregate: three of the four devices only ever supply one side of it. Which raises the obvious question — if they reach the same curve by different routes, what are those routes?

Overall mean load blurs two distinct factors: how long the machine runs vs. how hard it runs

$$\text{mean load} = \text{fraction of time running} \times \text{mean load while running}$$



● SW-065 — the on/off switch

$\text{corr}(\text{on}, T) +0.922$ · $\text{corr}(\text{load}|\text{on}, T) +0.531$

Load while running stays near 30 % at any temperature. Its positive correlation is entirely duty cycle: it is switched off in the cold, not running in reverse.

● ● SW-088 & SW-115 — both, same way

$\text{corr}(\text{on}, T) -0.925$ / -0.890 · $\text{corr}(\text{load}|\text{on}, T) -0.640$ / -0.818

They run more often and deliver more capacity as it gets colder. Clearly heating-dominated.

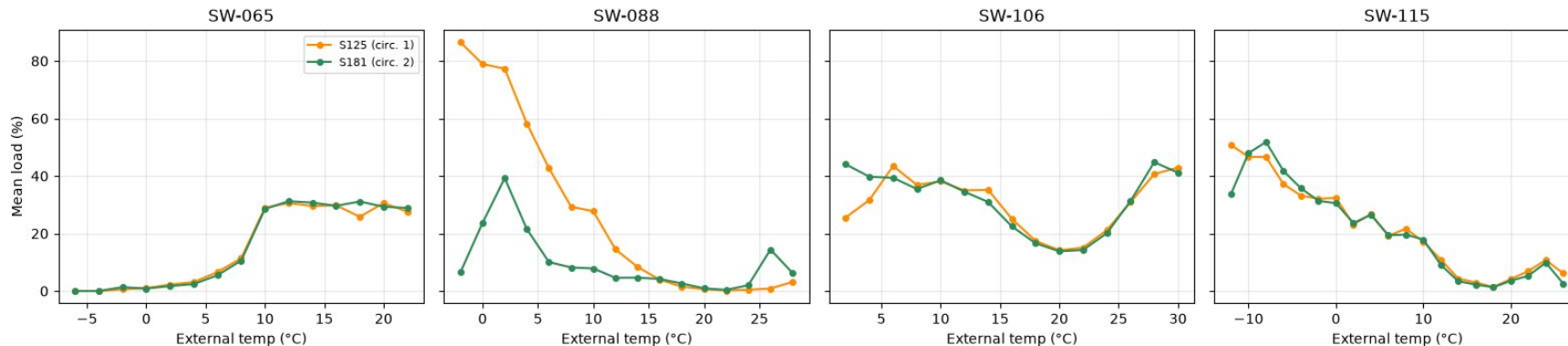
● SW-106 — the composite

$\text{corr}(\text{on}, T) -0.605$ · $\text{corr}(\text{load}|\text{on}, T) +0.848$

Duty cycle is V-shaped: it falls from cold to a minimum near 21 °C, then rises again in the heat. Load while running rises steadily with T. Below ~20 °C the two trends partly cancel; above it they add up.

The assignment names S125 and S181 separately, so it is worth asking whether they behave the same way.

Circuit 1 vs circuit 2, per device



One panel per device, on purpose

A pooled curve would flatten both circuits the same way and make them look similar for the wrong reason — SW-065 traces the profile backwards.

Three of four track each other

SW-065, SW-106 and SW-115 keep their two circuits on the same shape. SW-088 does not: circuit 1 reaches 87 % in the cold while circuit 2 stays under 40 %. That is the interesting case.

What it does not settle

Whether they work together or take turns — that depends on which minutes are counted.

corr(S125, S181) per device, over three different sets of minutes.

Device	all minutes	device running	both running
SW-088	0.333	-0.157	0.504
SW-106	0.575	0.058	0.264
SW-065	0.398	-0.329	-0.072
SW-115	0.677	0.257	0.602

Caution: for SW-088 the third column covers October to December only, because circuit 2 stops in January, so it is not directly comparable.

Column 1 is largely an artefact

53.6 % of minutes have both circuits at zero, so it mostly measures “they switch off together”.

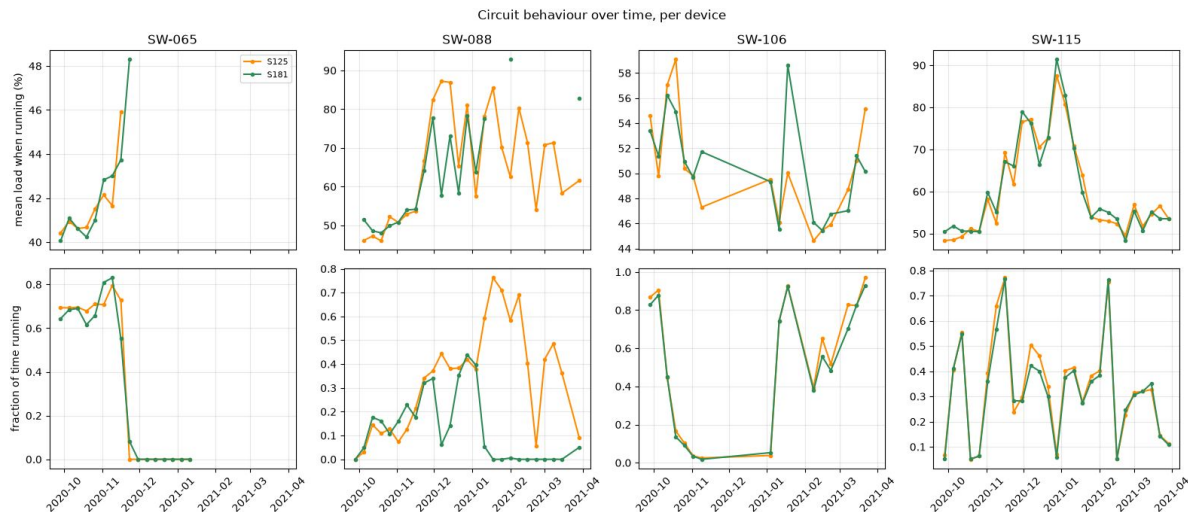
Remove the idle minutes and the sign flips

Three of four drop to zero or go negative. Negative means the circuits alternate.

Staging appears only in column 3

With both circuits genuinely on the coefficients turn positive: extra capacity really is needed and both are used.
Except for SW-065, the correlation drops near zero when both circuits run because the unit operates at a fixed load point around 30%.

Weekly behaviour of each circuit. Top row: mean load while running. Bottom row: fraction of time running.



SW-088 — circuit 2 stops reporting

First week of January. Circuit 1 keeps running and works harder ($\approx 55\% \rightarrow 60\text{--}85\%$). Gap std 12.7 pp against 1.0–3.1 elsewhere: a step change at a date, not a constant offset.

SW-065 — both circuits stop together

Early December. Stopping together points to a machine switched off, not a component failing, and it confounds its positive correlation with time.

What cannot be told apart for SW-088

A failed compressor, a failed sensor or a decommissioned circuit would all leave this same trace.

Neither uses the analysis above. One asks what the hardware can physically produce, the other what the machines are set to do:

1. Raw S125 and S181 readings, before any resampling

2. S123 and S179: circuit modes, columns the analysis never used.

1 · What values can a circuit produce?

Both OFF

0 %

Small ON

43 %

Large ON

57 %

Both ON

100 %

● SW-065	40 / 60	0 (149, 340) 40 (55, 562) 60 (6, 303) 100 (293)
● SW-088	34 / 66	0 (204, 684) 34 (24, 876) 66 (47, 216) 100 (46, 264)
● SW-106	43 / 57	0 (93, 615) 43 (59, 041) 57 (45, 474) 100 (2, 853)
● SW-115	43 / 57	0 (235, 640) 43 (38, 824) 57 (56, 321) 100 (23, 996)

Exactly four levels — two unequal on/off compressors

Raw readings yield exactly 4 discrete capacity levels (0%, 43%, 57%, 100%).
resampling to 1-minute intervals to compute correlations obscures the true, discrete load levels of the machine.

2 · What are the machines set to do?

Device	S123	S179	Active min.
SW-065	code 1 — 100 %	code 1 — 100 %	68,914
SW-088	code 4 — 81 %	code 4 — 86 %	14,779
SW-106	code 3 — 71 %	code 1 — 42 %	20,090
SW-115	code 4 — 82 %	code 4 — 92 %	39,947

Dominant codes over running minutes, Oct–Nov window only.

SW-065 never changes mode while running

Running minutes only (an idle machine cannot change mode) and the same Oct–Nov window for every device, because it's the only time interval where all devices are on.

Each lands where the deduction never reached: the raw sensor alphabet in one case, a separate set of columns in the other.

None of it is in the data, but the scalability section that follows is measured against it — which is why it comes now.

Element	Configuration
Machines	4 VMs, 2 cores each
Workers	3 VMs configured as workers → 6 cores in total
Master	hosts the cluster manager and the Driver
Executors	one per worker, 1 GB each
Storage	Parquet on an S3 bucket (CloudVeneto)
Cluster manager	Spark standalone

Each worker starts one executor, so asking for more cores also brings more memory ,1 GB at one core, 3 GB at six.

Why S3 and not a local file

A file on one node's disk is invisible to the executors running on the other two. Object storage is reachable from all of them, which is why the data path is s3a://.

The Driver runs in the Master

It builds the plan, splits it into tasks and talks to the executors continuously, so it has to sit inside the cluster network, in the notebook it is the Python kernel, in the benchmark it is the script.

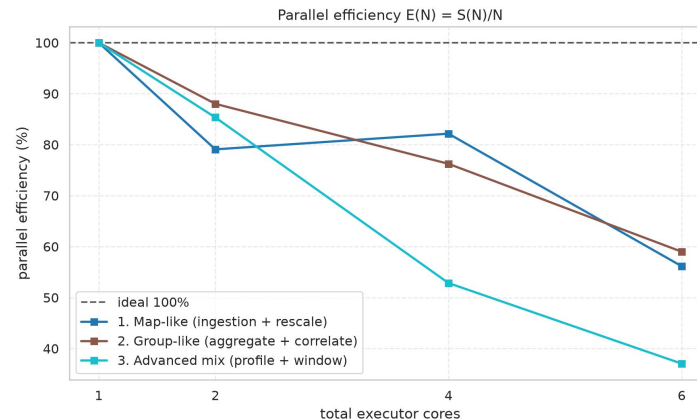
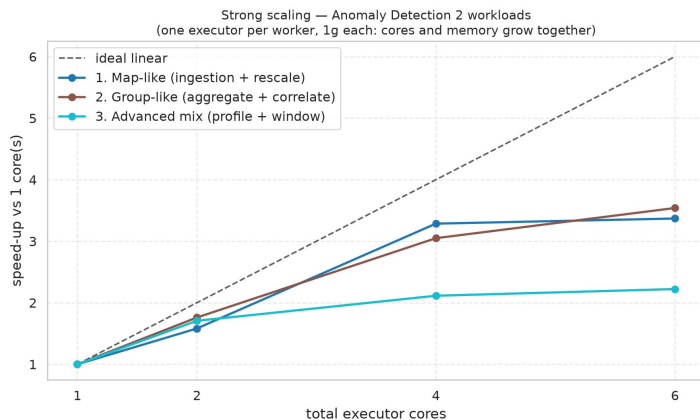
One Spark application per core count, cluster otherwise idle. Measured in a separate script, because a session's resources are fixed at creation.

Phase	What it does	Shape
1. Map-like	read, filter 3 metrics of ~120, rescale	narrow only, no shuffle
2. Group-like	resample, pivot, hourly aggregate, correlate	four wide dependencies
3. Advanced mix	binned profile, duty-cycle split, global sort	different keys, plus one stage that cannot parallelise

speed-up $S(p) = T(1) / T(p)$

efficiency $E(p) = S(p) / p$

ceiling $S(p) \leq 1 / f$



Amdahl's law, and the timings behind the three curves.

If a fraction f of the work is intrinsically serial, no number of cores can beat a speed-up of $1/f$. On top of that, coordination overhead grows with the cores.

Phase	1 core	2	4	6	Speed-up	Efficiency
1. Map-like	10.82 s	6.84 s	3.29 s	3.21 s	3.37×	0.56
2. Group-like	31.25 s	17.76 s	10.25 s	8.83 s	3.54×	0.59
3. Advanced mix	2.63 s	1.54 s	1.24 s	1.18 s	2.22×	0.37

Why it falls

Two reasons at once: the serial fraction puts a ceiling on speed-up, and coordination overhead grows with the number of cores. Efficiency of 1 would mean neither exists.

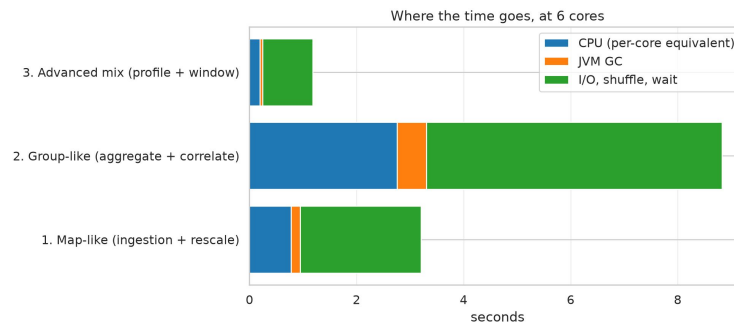
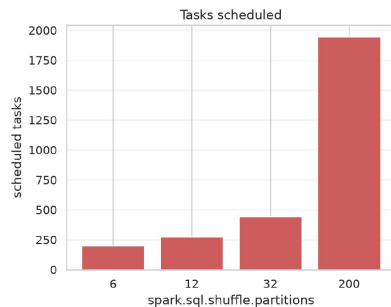
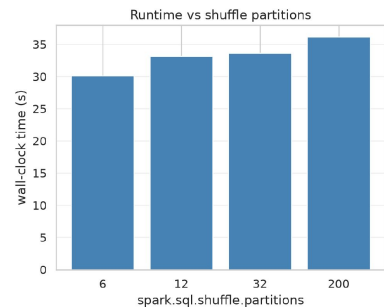
Why the mix falls furthest

It contains the global sort on purpose. If that phase scaled well, the benchmark would not be measuring what it claims to measure.

Why not more partitions

Tested at 6, 12, 32 and 200: runtime grows monotonically. With six cores, six partitions already saturate them.

Runtime against shuffle partitions at six cores, and where the time actually goes.



Runtime vs shuffle partitions

Runtime grows monotonically: 30.1 s at 6 partitions, 36.2 s at 200. Six cores are already saturated by six partitions.

Tasks scheduled

The reason: 194 tasks at six partitions against 1,944 at two hundred. Same six cores, ten times the scheduling.

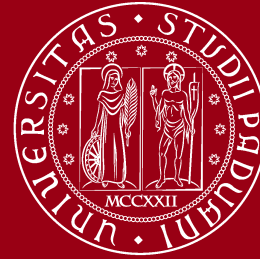
Where the time goes, at 6 cores

Most of the time is I/O, shuffle and waiting — not CPU. Adding cores adds compute, which is not the bottleneck.

The four profiles, with the numbers behind each claim.

	SW-065	SW-088	SW-106	SW-115
Pearson r vs S41	+0.767	-0.571	-0.269	-0.313
corr(running, T)	+0.922	-0.925	-0.605	-0.890
corr(load running, T)	+0.531	-0.640	+0.848	-0.818
Thermodynamic role	cooling-only (inferred)	heating-dominated	dual-role	heating-dominated
Staging split	40 / 60	34 / 66	43 / 57	43 / 57
Mean load (running)	30.8 %	44.7 %	40.9 %	48.0 %
p95 load	50 %	100 %	57 %	100 %
Time ≥ 90 %	~ 0 %	6.7 %	0.5 %	11.0 %
Dominant mode, S123	code 1 — 100 %	code 4 — 81 %	code 3 — 71 %	code 4 — 82 %
Asset health	winter shutdown	circuit 2 offline, Jan	healthy	healthy

Correlations against temperature are computed on hourly rows; the duty-cycle pair across temperature bins. Mode percentages are running minutes inside the Oct–Nov window.



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Predictive Maintenance: Industrial Engine Overheating Prediction

Distributed Telemetry Processing,
Physics-Informed Feature Engineering &
Operational Calibration

Problem Context & Objective

Industrial chillers and large-scale heat pumps face catastrophic motor winding burnout and fluid degradation caused by compressor overheating (Alarms A5 & A9).

Our goal is to build an early-warning classification system (T-1 → T) to predict impending overheating conditions using historical telemetry prior to failure.

Operational Mandate

- Near-Total Failure Interception: Guarantee detection during precursor windows before irreversible mechanical degradation occurs.
- Strict False-Alarm Prevention: Maintain absolute precision to avoid operator alarm fatigue and costly unneeded downtime.

Critical Data Imbalance Challenge

124

 Overheating Events Recorded

Across **671,000 aggregated operational minutes** over 6 months of continuous monitoring, only 124 positive fault instances occurred.

Class Probability: $p \approx 0.0185\%$ (~1 fault per 5,400 normal minutes)

Monitoring Period

6 Months

671,000 total minutes of operational telemetry data

Fault Frequency

1 : 5,400

Extreme minority ratio requiring specialized modeling

STAGE 01

1

Native Bitwise Alarm Decoding

REGISTER COMPRESSION

Packs 16 boolean sensor alarms into a single 16-bit integer register (**A5 & A9**) to minimize bandwidth.

BITWISE MASKING

Direct CPU masking evaluates **bits 6, 7, and 8** to instantly isolate active overheating flags.

STAGE 02

2

1-Min Resampling & Physics Aggregation

TEMPORAL STANDARDIZATION

Asynchronous sensor pings are standardized into **1-minute discrete temporal grids**.

PHYSICS AGGREGATION

- **Continuous:** Mean aggregation filters signal jitter.
- **Discrete:** Max aggregation preserves transient spikes.

STAGE 03

3

OOM-Safe Wide Pivoting & Imputation

MEMORY-SAFE PIVOTING

To prevent driver out-of-memory crashes, **30 thermodynamic metrics** are pivoted via conditional aggregations instead of full Spark cross-tab shuffles.

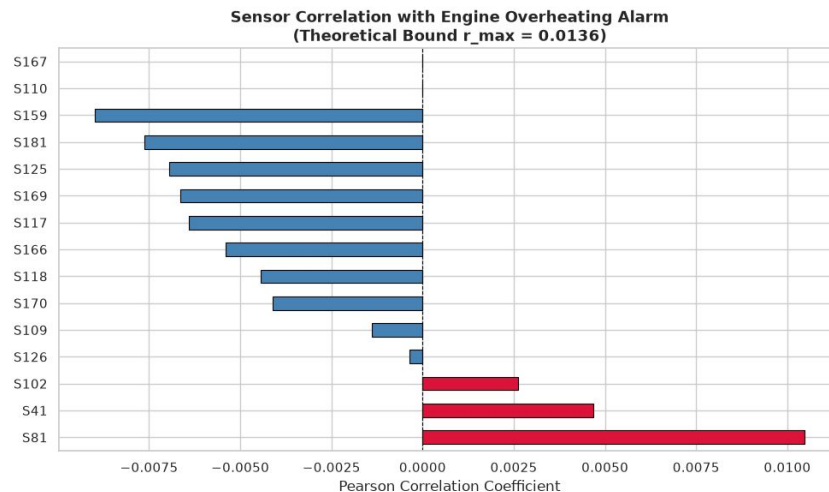
FORWARD-FILL IMPUTATION

Chronological forward-fill is applied within device partitions with **zero future lookahead**.

Superheat (S81) exhibits the highest positive correlation ($r = +0.0105$), indicating insufficient suction gas density to effectively cool hermetic motor windings.

Ambient Temperature (S41) increases condensing pressure and overall mechanical workload, acting as a key secondary driver.

Mathematical Bound: Due to extreme class imbalance ($p = 0.000185$), the theoretical maximum Pearson correlation is bounded at $r_{\max} \approx 0.0136$. The observed value proves strong physical relevance despite statistically small absolute magnitude.



77.1%

Theoretical Ceiling Reached

Superheat correlation (S81) achieves 77.1% of the mathematical limit $r_{\max} \approx 0.0136$.

+51.9%

RELATIVE RISE

Superheat (S81) Surge

Jumps sharply from **73.57°C** under normal operation to **111.78°C** during overheating.

+21.3%**Ambient Temp (S41)**

Simultaneous rise from 105.37 to 127.81.

Thermodynamic State Coupling

- **Discharge Pressures (S102):** Demonstrates persistent elevation prior to thermal cutoff.
- **Circuit 2 Depressurization / Cut Off:** The concurrent plunge in Circuit 2 discharge pressure (S159, -32.1%) and temperature (S166, -28.0%) highlights protective engine shedding/shutdown in response to the thermal emergency.

Sensor	Normal (0) Mean	Overheated (1) Mean	Abs Diff	Rel Change (%)
S109	442.14	412.22	-29.92	-6.77%
S166	368.42	265.28	-103.14	-27.99%
S102	183.80	199.20	+15.41	+8.38%
S159	164.96	111.93	-53.03	-32.15%
S126	21.96	21.63	-0.33	-1.50%
S81	73.57	111.78	+38.21	+51.94%
S41	105.37	127.81	+22.44	+21.29%

$r \approx 0.92$
CIRCUIT COUPLING

Strong Circuit Coupling

Near-perfect physical collinearity between discharge temperature, pressure, and compression ratio.

$|r| < 0.01$
IMBALANCE EFFECT

Class Imbalance Effect

Direct linear correlation with target is artificially bounded by extreme rarity ($p = 0.0185\%$).

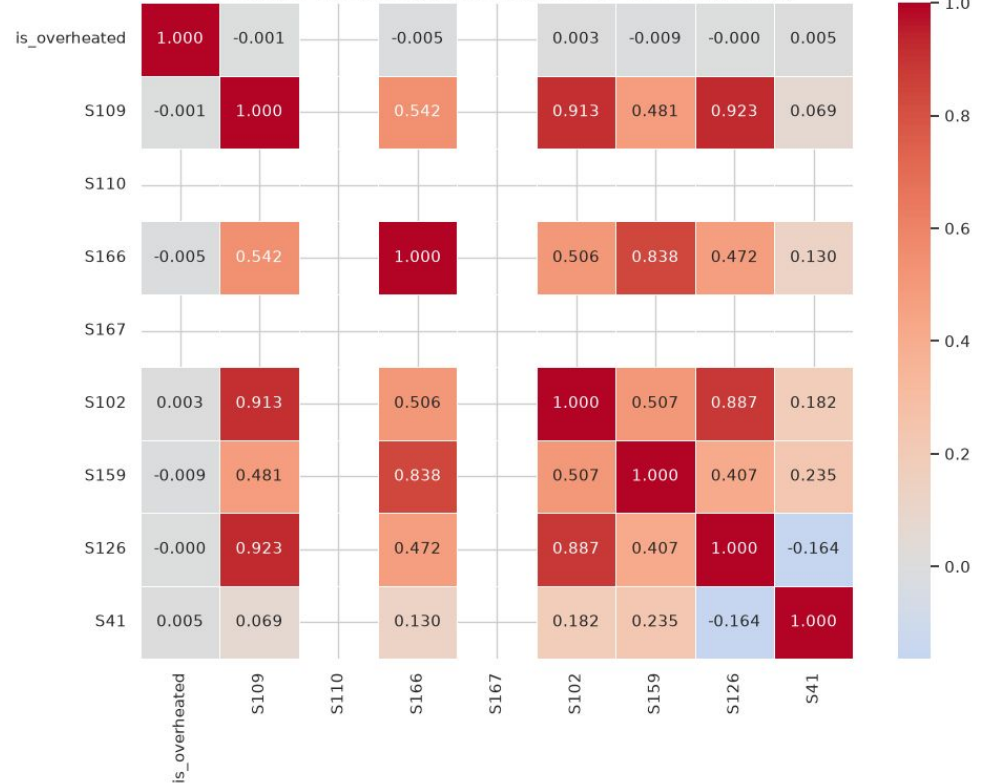
Thermodynamic State Coupling

- **Circuit 1 In-Lockstep ($r = 0.89 - 0.92$):** Discharge temperature (S109), pressure (S102), and compression ratio (S126) shift simultaneously, validating real gas compression dynamics.
- **Circuit 2 Alignment ($r = 0.84$):** S166 and S159 show strong pairwise tracking during operational changes.

Statistical & ML Insights

- **The Pearson Paradox:** Near-zero target correlation is a mathematical artifact ($r_{\max} = 0.0136$), not a lack of physical predictive power.
- **Zero-Variance Pruning:** S110 and S167 have null variance ($\sigma = 0$, blank bands) and are safely removed.
- **Multicollinearity Mitigation:** High cross-sensor redundancy ($r > 0.85$) motivates engineered dynamic features (e.g., 5-min gradients $\Delta 5\text{min}$) over raw correlated inputs.

Heatmap: Thermodynamic Indicators vs Overheating



Strict Causal Lagging (T-1 to T)

Zero Target Leakage Observation

Features are observed strictly at **lag 1** and **lag 5** relative to the prediction timestamp to ensure zero target leakage.

Dynamic Thermal Derivatives

Engineered **5-minute dynamic derivatives** to capture rapid thermal build-up before safety triggers activate.

Operational State Boundary

Compressor operational status is evaluated **exclusively at T-1** to prevent inadvertent temporal contamination.

Defrost Dynamics & Harmonic Encoding

⚠ Nocturnal Defrost Transients (SW-088)

Overheating alarms on device **SW-088** systematically cluster between **00:00 and 02:00** during automated winter defrost cycles.

This systematic cluster requires explicit temporal modeling to avoid false anomaly classification.

🕒 Deterministic Calendar Encoding

Encoded circular daily time using **harmonic pairs** and an **active defrost window flag**.

Utilizes strictly deterministic calendar knowledge available at **T-1**, ensuring complete consistency without future leakage.

Standard random CV is invalid due to strong temporal autocorrelation & defrost regime shifts.

6-Month Observation Horizon Split (~18.2 Days / Block)

■ Train (7)

■ Test (3)

▨ 60m Embargo



60-Minute Purging & Embargo Mechanism

To eliminate lag cross-over and thermal inertia leakage, the **first 60 minutes of each test block** adjacent to the training set were completely discarded.

Train Partition Details

496,575

Training Samples

71

Fault Events Yielded

- Allocated **7 contiguous chronological blocks** (~127.4 days total).
- Captures baseline operating dynamics and seasonal temperature trends across the observation period.

Test Partition & Purging Details

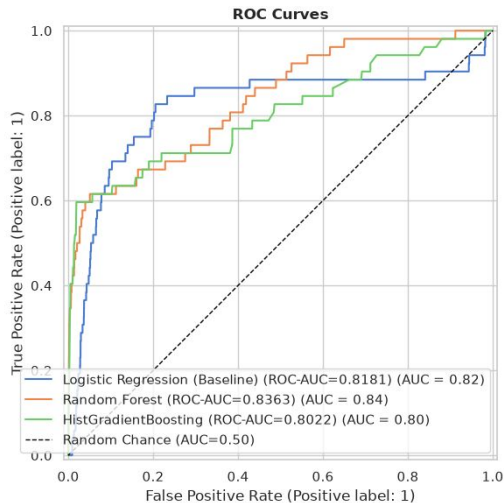
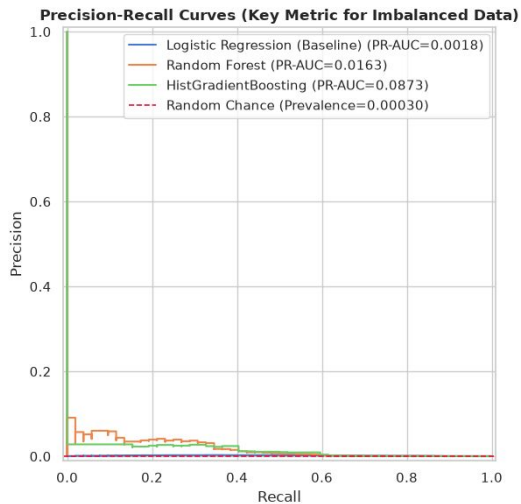
174,831

Test Samples (Post-Purge)

52

Fault Events Yielded

- Allocated **3 contiguous chronological blocks** (~54.6 days total).
- Embargo strategy successfully isolated winter defrost regime shifts without target or feature leakage.



Imbalance Distortion: With 5,000:1 negative-to-positive ratio, ROC-AUC remains artificially inflated (~0.802) by massive true negatives. PR-AUC directly reveals true precision performance.

~291x

PR-AUC Improvement Over Baseline
HistGradientBoosting (0.0873 vs 0.0003 prevalence)

MODEL ARCHITECTURE	PR-AUC	ROC-AUC
HistGradBoost	0.0873	0.802
Random Forest	0.0163	0.836
Logistic Regression	0.0018	0.818



CAPTURED COMPLEX INTERACTIONS

Non-linear models outperform linear baselines by capturing multi-variable physical dependencies:

• Superheat Elevation • Ambient Temp • Defrost Hours

Standard threshold optimization selects extreme values, reducing operational recall. We define three calibrated modes to balance high-sensitivity failure interception with precision-focused false alarm reduction.



DEFAULT BALANCED

50.0%

POINT RECALL

1.45% Active Alarm Rate

Provides a baseline trade-off for standard operating conditions. Balances operational detection capabilities with a manageable baseline volume of active system alerts.



HIGH SENSITIVITY

65.4%

POINT RECALL

Mission-Critical Focus

Specifically targets mission-critical plants where fault interception is paramount. Maximizes recall performance to successfully prevent expensive motor replacements and severe damage.



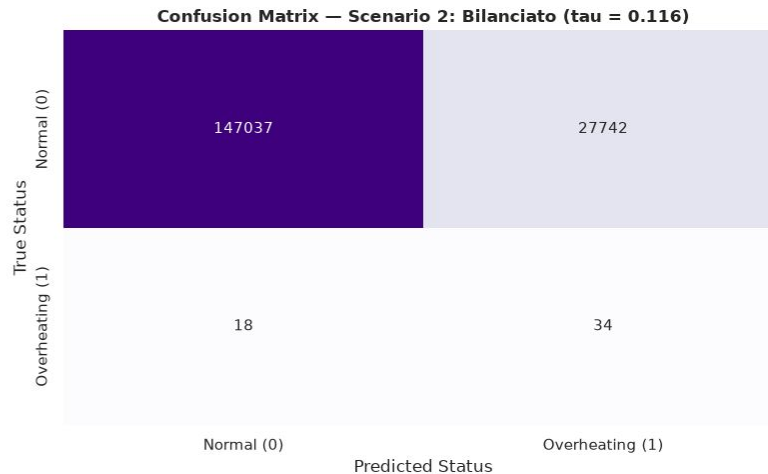
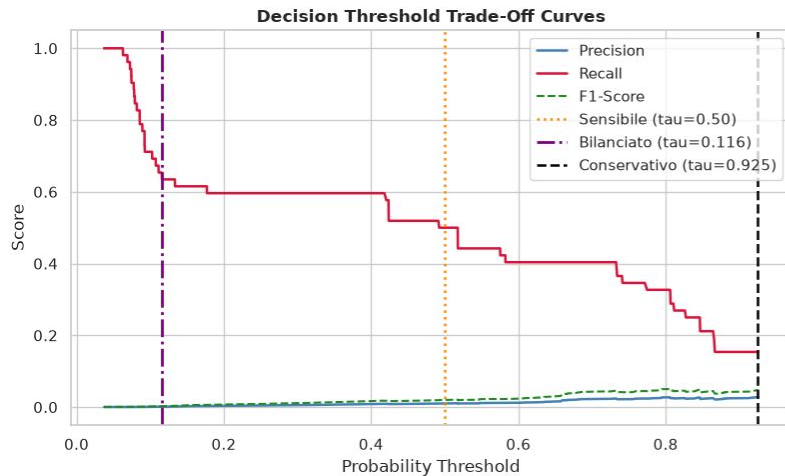
CONSERVATIVE

0.16%

ACTIVE ALARM RATE

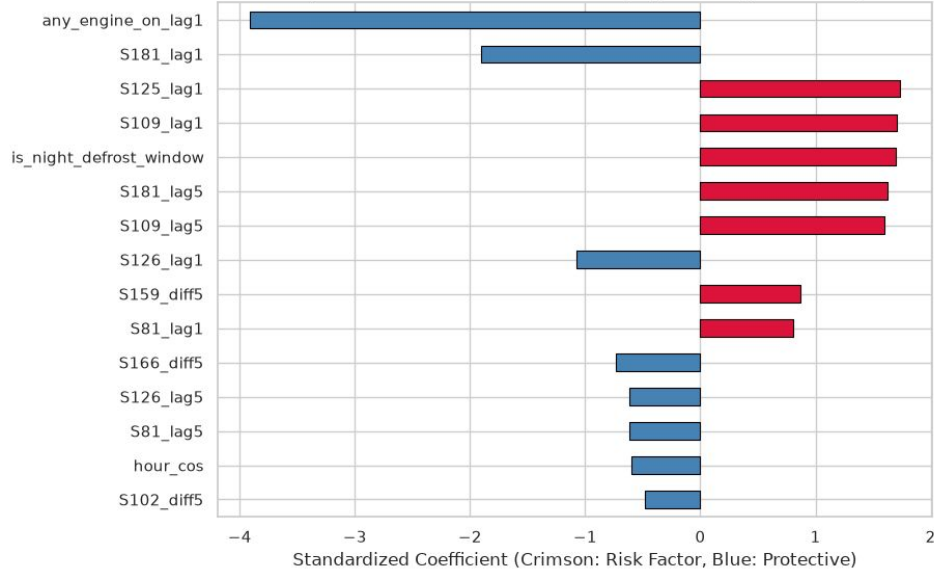
-89.0% False Alarms

Slashes false alarms down to a minimal 0.16% active rate. Designed to effectively prevent operator fatigue while maintaining maximum precision across non-critical monitoring environments.

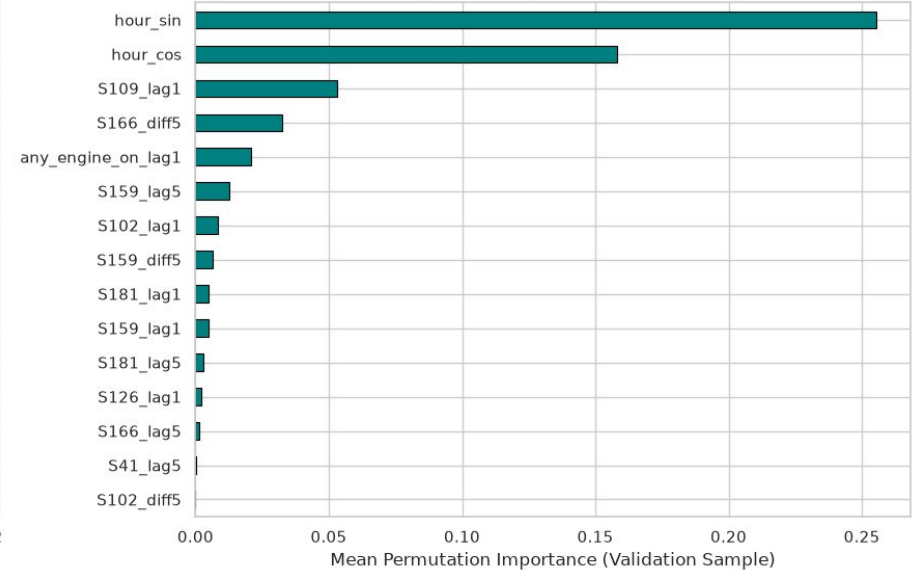


#	Operational Scenario	Threshold ()	Precision	Recall	F1-Score	False Alarms (FP)	Alarm Active Rate (%)	Episodes Detected
0	Scenario 1: Sensible (=0.50)	0.5000	0.0103	0.5000 (50.0%)	0.0201	2,508	1.45%	20 / 33 (60.6%)
1	Scenario 2: Balanced (=0.116, Rec>=65%)	0.1156	0.0012	0.6538 (65.4%)	0.0024	27,742	15.89%	31 / 33 (93.9%)
2	Scenario 3: Conservative (=0.925, F0.5)	0.9252	0.0282	0.1538 (15.4%)	0.0476	276	0.16%	9 / 33 (27.3%)

Top 15 Standardized Coefficients (Logistic Regression L2)



Top 15 Permutation Importances (HistGradientBoosting - ROC-AUC Drop)



DISTRIBUTED PYSPARK MLLIB (30 TREES)

ROC-AUC: 0.769 | PR-AUC: 0.0034

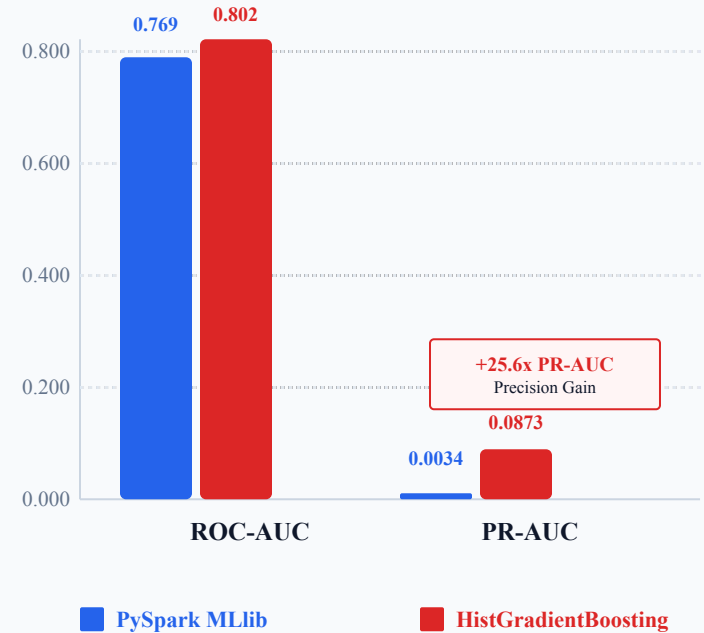
Indispensable for distributed heavy-lifting, including terabyte-scale Parquet ingestion, 1-minute temporal resampling, and wide pivoting across cluster nodes.

ARCHITECTURAL TAKEAWAY

Spark is essential for distributed ETL & pivoting of raw telemetry; once aggregated into RAM, single-node boosting eliminates cluster communication overhead while providing advanced cost-sensitive weighting for extreme imbalance.

PERFORMANCE BENCHMARK COMPARISON

PySpark MLib vs. Local HistGradientBoosting



EXECUTION TIME BENCHMARK

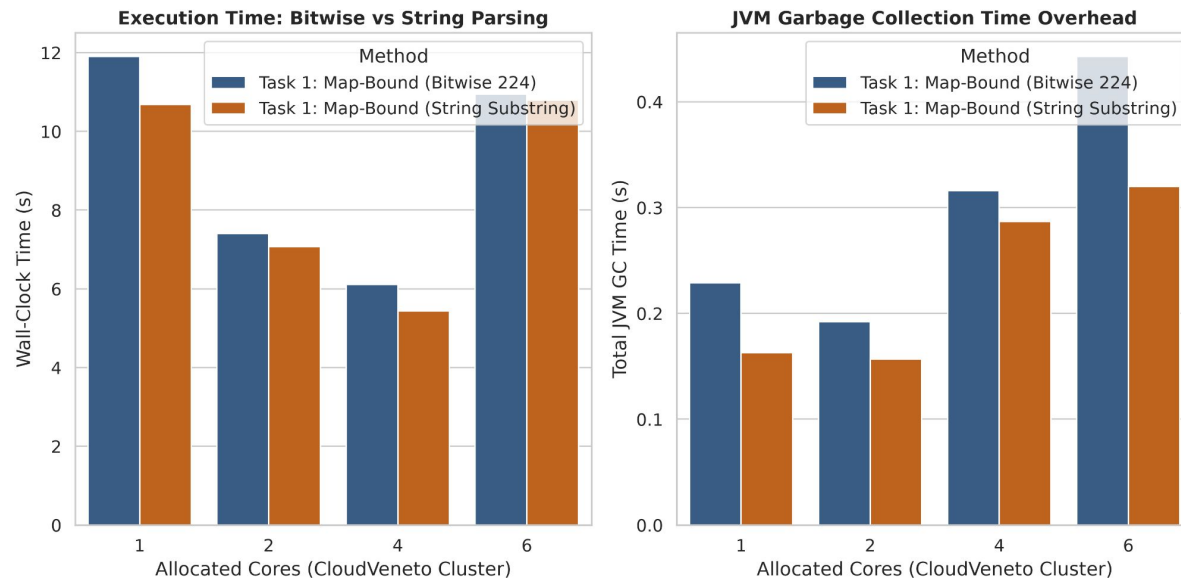
Optimal Scale at 4 Cores

Bitwise 224 and String Substring show effective speedup from 1 to 4 cores (~5.5s minimum). Scaling to 6 cores degrades wall-clock performance back to ~11s due to core contention.

GARBAGE COLLECTION OVERHEAD

GC Penalty Spikes at 6 Cores

Total JVM GC time remains low (<0.2s) up to 2 cores, but rises steeply at 4 and 6 cores (>0.4s). Bitwise parsing consistently incurs higher GC overhead than String Substring.



STRONG SCALING SPEEDUP

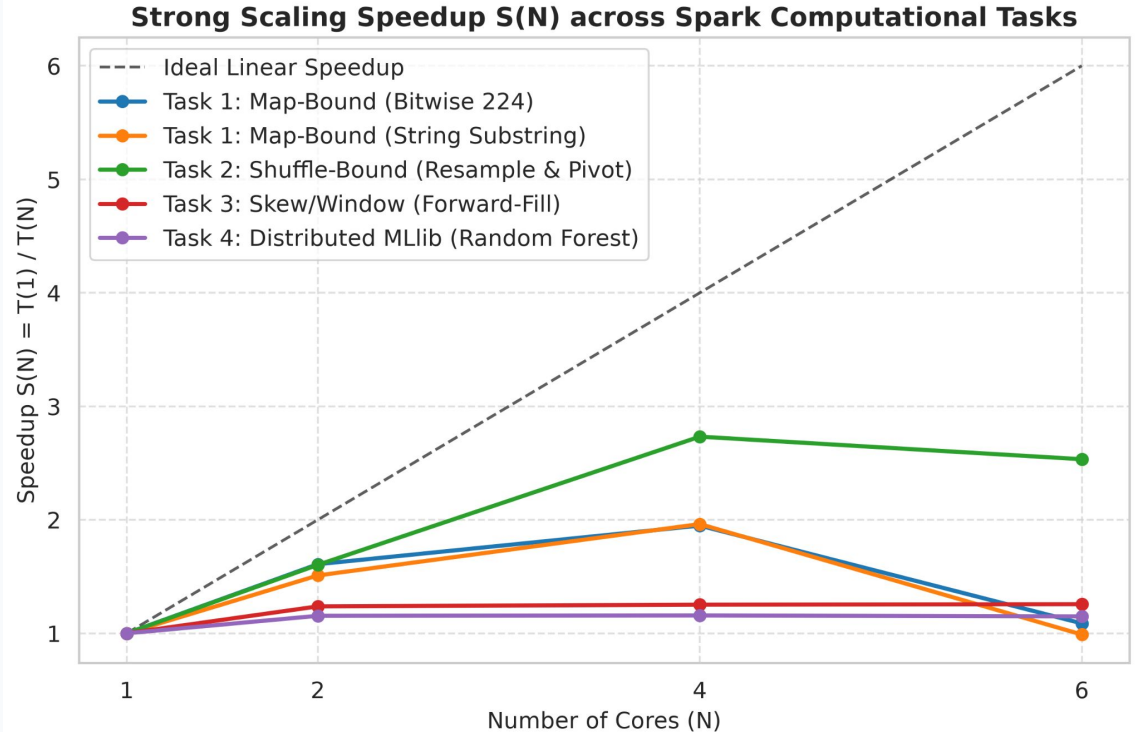
Shuffle-Bound Task Scales Best (2.73x)

Task 2 (Resample & Pivot) achieves the highest speedup, dropping from 151.1s (1 core) to 55.3s (4 cores) as its massive 1.27 GB shuffle volume amortizes cluster overhead. Scaling plateaus at 6 cores (~59.6s) due to virtualized inter-VM network limits.

AMDAHL'S LAW & 6-CORE BOTTLENECK

All Tasks Degradate at 6 Cores

Scaling to 6 cores brings no speedup across any workload: Task 1 degrades due to driver RPC overhead on 3 VMs; Task 2 plateaus due to 1.27 GB network shuffle limits; Task 3 is capped at 4 cores by device cardinality (hwid); and MLlib Random Forest is bounded by fixed driver split synchronization (~5.5s).



PARALLEL EFFICIENCY BENCHMARK

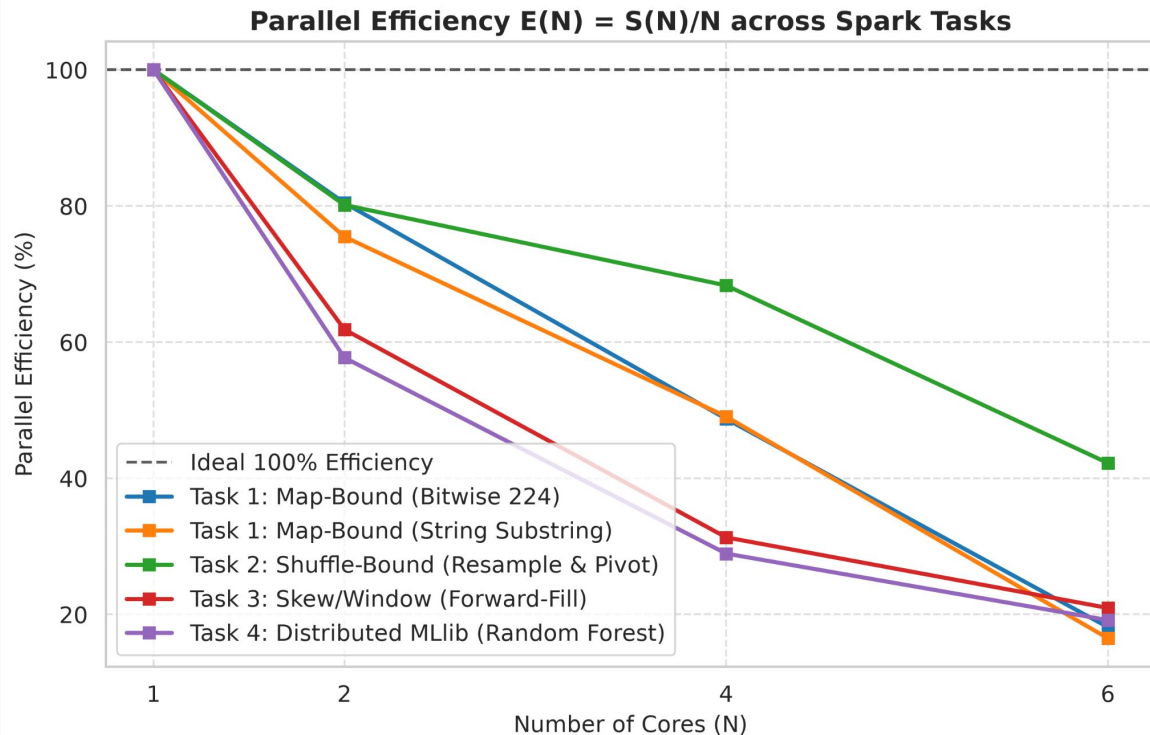
Heavy Tasks Retain Highest Efficiency

Parallel efficiency decreases across all archetypes as cores increase, reflecting Amdahl's Law. Task 2 maintains superior efficiency (80% at 2 cores, 68% at 4 cores, 42% at 6 cores), demonstrating that compute- and data-heavy workloads benefit most from parallelism.

DISTRIBUTED OVERHEAD PENALTY

Severe Collapse for Fine-Grained Tasks

Lightweight and coordination-heavy workloads suffer sharp efficiency drops at 6 cores (Task 1: 18%, Task 3: 21%, Task 4: 19%). Beyond 4 cores, task scheduling, network latency, and driver coordination dominate over useful computation.



SHUFFLE PARTITIONS OPTIMIZATION

Monotonic Runtime Gain up to 200 Partitions

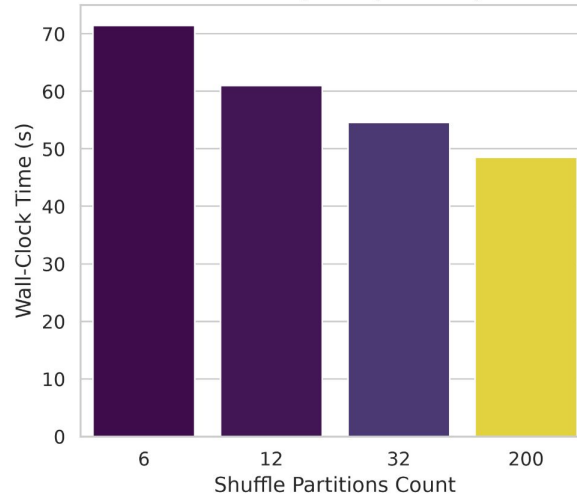
Increasing shuffle partitions from 6 to 200 monotonically reduces pivot duration from 71.4s to 48.5s. Coarse partitions ($p=6$) cause memory pressure and GC spikes, whereas fine partitions split the 1.27 GB shuffle into smaller, smoother pipeline blocks.

TASK SCHEDULING TRADEOFF

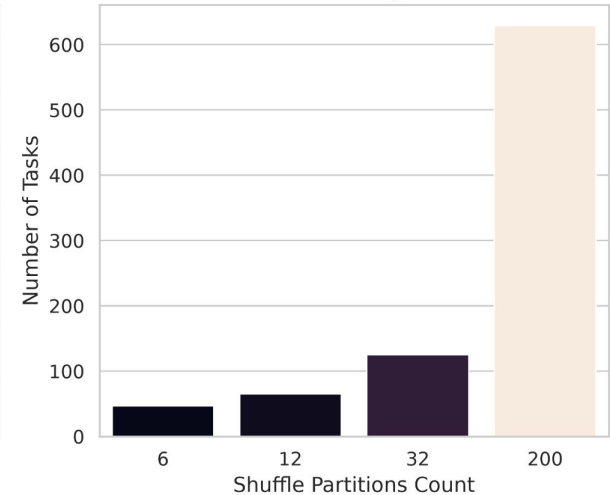
32 Partitions as the Pragmatic Sweet Spot

Although 200 partitions yields the fastest execution, it generates 629 tasks, increasing driver scheduling overhead. Tuning to 32 partitions slashes task count by 80% (125 tasks) with only a minor ~6s runtime penalty (54.6s).

Execution Time vs spark.sql.shuffle.partitions



Total Scheduled Spark Tasks



Execution Time Breakdown at 6 Cores (CloudVeneto Cluster)